```
!pip install qiskit qiskit-aer qiskit-paulice matplotlib numpy
```

```
Collecting qiskit
  Downloading qiskit-2.6.1-cp310-abi3-manylinux_2_28_x86_64.whl.metadata (14 kB)
Collecting qiskit-aer
  Downloading qiskit_aer-0.18.2-cp313-cp313-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (8.7 kB)
Collecting qiskit-paulice
  Downloading qiskit_paulice-0.3-cp310-abi3-manylinux_2_28_x86_64.whl.metadata (12 kB)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.13/dist-packages (3.10.0)
Requirement already satisfied: numpy in /usr/local/lib/python3.13/dist-packages (2.1.3)
Requirement already satisfied: scipy>=1.14 in /usr/local/lib/python3.13/dist-packages (from qiskit) (1.16.3)
Collecting rustworkx>=0.15.0 (from qiskit)
  Downloading rustworkx-0.18.1-cp310-abi3-manylinux2014_x86_64.manylinux_2_17_x86_64.whl.metadata (10 kB)
Requirement already satisfied: dill>=0.3 in /usr/local/lib/python3.13/dist-packages (from qiskit) (0.4.1)
Collecting stevedore>=3.0.0 (from qiskit)
  Downloading stevedore-5.9.1-py3-none-any.whl.metadata (2.3 kB)
Requirement already satisfied: typing-extensions in /usr/local/lib/python3.13/dist-packages (from qiskit) (4.16.0)
Requirement already satisfied: psutil>=5 in /usr/local/lib/python3.13/dist-packages (from qiskit-aer) (5.9.5)
Requirement already satisfied: python-dateutil>=2.8.0 in /usr/local/lib/python3.13/dist-packages (from qiskit-aer) (2.9.0.post
Collecting samplomatic>=0.20.0 (from qiskit-paulice)
  Downloading samplomatic-0.21.0-py3-none-any.whl.metadata (6.7 kB)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.13/dist-packages (from matplotlib) (1.4.0)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.13/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.13/dist-packages (from matplotlib) (4.67.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.13/dist-packages (from matplotlib) (1.5.1)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.13/dist-packages (from matplotlib) (26.3)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.13/dist-packages (from matplotlib) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.13/dist-packages (from matplotlib) (3.3.3)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.13/dist-packages (from python-dateutil>=2.8.0->qiskit-aer) (
Collecting pybase64>=1.4.0 (from samplomatic>=0.20.0->qiskit-paulice)
  Downloading pybase64-1.5.1-cp313-cp313-manylinux1_x86_64.manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_5_x86_64.whl
Requirement already satisfied: orjson>=3.9.0 in /usr/local/lib/python3.13/dist-packages (from samplomatic>=0.20.0->qiskit-paul
Downloading qiskit-2.6.1-cp310-abi3-manylinux_2_28_x86_64.whl (9.6 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 9.6/9.6 MB 76.0 MB/s  0:00:00
Downloading qiskit_aer-0.18.2-cp313-cp313-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (14.2 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 14.2/14.2 MB 114.5 MB/s  0:00:00
Downloading qiskit_paulice-0.3-cp310-abi3-manylinux_2_28_x86_64.whl (656 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 656.0/656.0 kB 16.8 MB/s  0:00:00
Downloading rustworkx-0.18.1-cp310-abi3-manylinux2014_x86_64.manylinux_2_17_x86_64.whl (2.4 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2.4/2.4 MB 56.8 MB/s  0:00:00
Downloading samplomatic-0.21.0-py3-none-any.whl (234 kB)
Downloading pybase64-1.5.1-cp313-cp313-manylinux1_x86_64.manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_5_x86_64.whl (
Downloading stevedore-5.9.1-py3-none-any.whl (54 kB)
Installing collected packages: stevedore, rustworkx, pybase64, qiskit, samplomatic, qiskit-aer, qiskit-paulice
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 7/7 [qiskit-paulice]
Successfully installed pybase64-1.5.1 qiskit-2.6.1 qiskit-aer-0.18.2 qiskit-paulice-0.3 rustworkx-0.18.1 samplomatic-0.21.0 st
```

New Section

```
import qiskit
import qiskit_aer
import qiskit_paulice
import numpy as np
import matplotlib.pyplot as plt

print("Qiskit version:", qiskit.__version__)
print("Qiskit Aer imported successfully")
print("Qiskit Paulice imported successfully")
print("All libraries are ready!")
```

```
Qiskit version: 2.6.1
Qiskit Aer imported successfully
Qiskit Paulice imported successfully
All libraries are ready!
```

```
from qiskit import QuantumCircuit

# Create a circuit with 3 qubits
qc = QuantumCircuit(3)

# Apply Hadamard gates
qc.h(0)
qc.h(1)
```

```
# Entangle the qubits
qc.cx(0, 1)
qc.cx(1, 2)

# Apply phase gates
qc.s(0)
qc.s(1)

# Display circuit
print(qc.draw())
```

```
     ┌───┐     ┌───┐
q_0: ┤ H ├──■──┤ S ├─────
     ├───┤  │  └───┘
q_1: ┤ H ├──X──■──┤ S ├──
     └───┘     │  └───┘
q_2: ──────────┤ X ├─────
               └───┘
```

```
from qiskit import QuantumCircuit

# Create a 3-qubit T-doped circuit
tqc = QuantumCircuit(3)

# Step 1: Superposition
tqc.h(0)
tqc.h(1)

# Step 2: Entanglement
tqc.cx(0, 1)
tqc.cx(1, 2)

# Step 3: Add T gates
tqc.t(0)
tqc.t(1)
tqc.t(2)

# Display circuit
print(tqc.draw())
```

```
     ┌───┐     ┌───┐
q_0: ┤ H ├──■──┤ T ├─────
     ├───┤  │  └───┘
q_1: ┤ H ├──X──■──┤ T ├──
     └───┘     │  └───┘
q_2: ──────────┤ X ├─────
               └───┘
```

```
print("Clifford circuit:")
print(qc.draw())

print("\nT-doped circuit:")
print(tqc.draw())

print("\nNumber of qubits:", tqc.num_qubits)
print("Number of T gates:", tqc.count_ops().get('t', 0))
```

```
Clifford circuit:
     ┌───┐     ┌───┐
q_0: ┤ H ├──■──┤ S ├─────
     ├───┤  │  └───┘
q_1: ┤ H ├──X──■──┤ S ├──
     └───┘     │  └───┘
q_2: ──────────┤ X ├─────
               └───┘

T-doped circuit:
     ┌───┐     ┌───┐
q_0: ┤ H ├──■──┤ T ├─────
     ├───┤  │  └───┘
q_1: ┤ H ├──X──■──┤ T ├──
     └───┘     │  └───┘
q_2: ──────────┤ X ├─────
               └───┘

Number of qubits: 3
Number of T gates: 3
```

```
print("===== CLIFFORD CIRCUIT =====")
print(qc.draw())

print("\n===== T-DOPED CIRCUIT =====")
print(tqc.draw())

print("\n===== CIRCUIT STATISTICS =====")
print("Clifford qubits:", qc.num_qubits)
print("T-doped qubits:", tqc.num_qubits)
print("T gates:", tqc.count_ops().get('t', 0))
```

```
===== CLIFFORD CIRCUIT =====
     ┌───┐     ┌───┐
q_0: ┤ H ├──■──┤ S ├─────
     ├───┤┌─┴─┐├───┤┌───┐
q_1: ┤ H ├┤ X ├──■──┤ S ├
     └───┘└───┘┌─┴─┐└───┘
q_2: ──────────┤ X ├─────
               └───┘

===== T-DOPED CIRCUIT =====
     ┌───┐     ┌───┐
q_0: ┤ H ├──■──┤ T ├─────
     ├───┤┌─┴─┐└───┘┌───┐
q_1: ┤ H ├┤ X ├──■──┤ T ├
     └───┘└───┘┌─┴─┐└───┘
q_2: ──────────┤ X ├─────
               └───┘

===== CIRCUIT STATISTICS =====
Clifford qubits: 3
T-doped qubits: 3
T gates: 3
```

```
from qiskit import QuantumCircuit

# Create a fresh Clifford payload circuit
clifford_qec = QuantumCircuit(3)

clifford_qec.h(0)
clifford_qec.h(1)

clifford_qec.cx(0, 1)
clifford_qec.cx(1, 2)

clifford_qec.s(0)
clifford_qec.s(1)

# Add terminal measurements
clifford_qec.measure_all()

print(clifford_qec.draw())
```

```
     ┌───┐     ┌───┐      ┌─┐
q_0: ┤ H ├──■──┤ S ├──────┤M├──────
     ├───┤┌─┴─┐├───┤┌───┐ └╥┘┌─┐
q_1: ┤ H ├┤ X ├──■──┤ S ├──╫─┤M├──
     └───┘└───┘┌─┴─┐└───┘  ║ └╥┘┌─┐
q_2: ──────────┤ X ├───────╫──╫─┤M├
               └───┘       ║  ║ └╥┘
meas: 3/═══════════════════╩══╩══╩═
                           0  1  2
```

```
from qiskit_paulice.noise_models import NoiseModel

# Simulated noise parameters
noise_model = NoiseModel(
    gate_noise=0.001,
    readout_noise=0.01
)

print("Noise model created successfully!")
print(noise_model)
```

```
Noise model created successfully!
NoiseModel(gate_noise=0.001, readout_noise=0.01, idling_noise=None)
```

```
from qiskit_paulice import add_pauli_checks

# Select the three data qubits
target_qubits = [0, 1, 2]

# Generate spacetime checks
checked_variants = add_pauli_checks(
    clifford_qec,
    target_qubits,
    noise_model,
    cost="gamma",
    method="windowed",
    seed=42
)

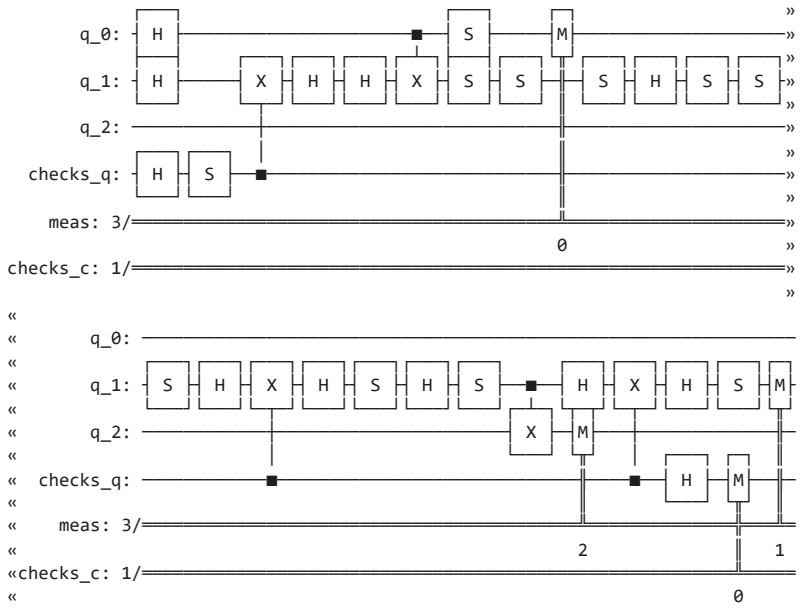
# Select the circuit with the maximum number of generated checks
checked_circuit = checked_variants[-1]

print("Number of circuit variants:", len(checked_variants))
print("Number of checks:", len(checked_circuit.check_qubits))

print("\nChecked quantum circuit:")
print(checked_circuit.circuit.draw())
```

```
Number of circuit variants: 2
Number of checks: 1

Checked quantum circuit:
```




```
from qiskit_paulice import add_pauli_checks

# Select the three data qubits
target_qubits = [0, 1, 2]

# Generate spacetime checks
checked_variants = add_pauli_checks(
    clifford_qec,
    target_qubits,
    noise_model,
    cost="gamma",
    method="windowed",
    seed=42
)

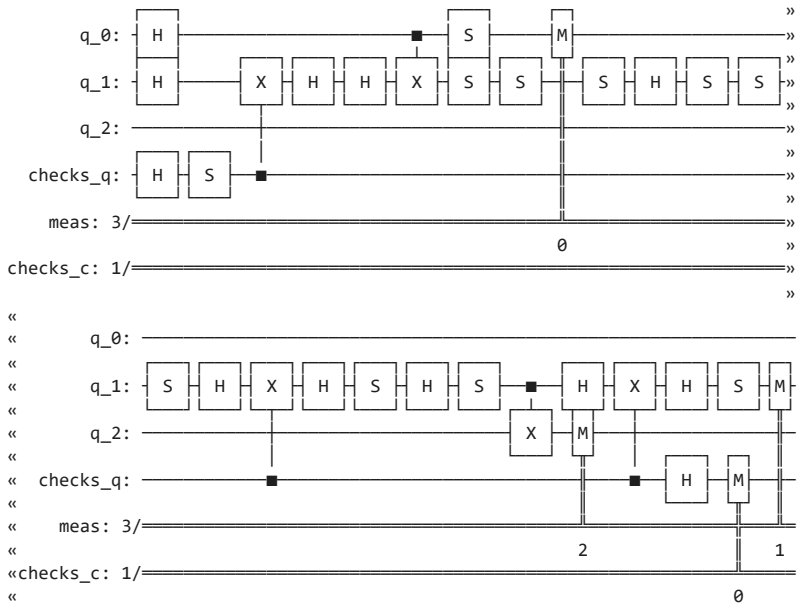
# Select the circuit with the maximum number of generated checks
checked_circuit = checked_variants[-1]

print("Number of circuit variants:", len(checked_variants))
print("Number of checks:", len(checked_circuit.check_qubits))

print("\nChecked quantum circuit:")
print(checked_circuit.circuit.draw())
```

```
Number of circuit variants: 2
Number of checks: 1

Checked quantum circuit:
```




```
from qiskit import transpile
from qiskit_aer import AerSimulator
from qiskit_aer.noise import (
    NoiseModel as AerNoiseModel,
    depolarizing_error,
    ReadoutError
)

# Create Aer noise model
aer_noise = AerNoiseModel()

# Two-qubit gate noise
gate_error = depolarizing_error(
    noise_model.gate_noise, 2
)

aer_noise.add_all_qubit_quantum_error(
    gate_error, ["cx"]
)

# Measurement noise
p = noise_model.readout_noise

readout_error = ReadoutError([
    [1-p, p],
    [p, 1-p]
])

aer_noise.add_all_qubit_readout_error(readout_error)

# Create simulator
simulator = AerSimulator(
    method="stabilizer",
    noise_model=aer_noise
)

# Transpile circuit
compiled_circuit = transpile(
    checked_circuit.circuit,
    simulator,
    optimization_level=0
)

# Run simulation
result = simulator.run(
    compiled_circuit,
    shots=1000,
```

```
    seed_simulator=42
).result()

counts = result.get_counts()

print("Total shots:", sum(counts.values()))
print("Measurement results:")
print(counts)
```

```
Total shots: 1000
Measurement results:
{'1 001': 1, '1 000': 4, '0 101': 9, '0 100': 7, '1 101': 1, '0 110': 243, '1 111': 4, '0 001': 239, '0 011': 2, '0 111': 240,
```

```
# Get post-selection function
ps_fn = checked_circuit.get_postselection_method()

# Keep only shots with zero syndrome
counts_postselected = {
    bitstring: number
    for bitstring, number in counts.items()
    if not ps_fn(bitstring).any()
}

total = sum(counts.values())
kept = sum(counts_postselected.values())
rejected = total - kept

acceptance_rate = (kept / total) * 100

print("Total shots:", total)
print("Accepted shots:", kept)
print("Rejected shots:", rejected)

print("Acceptance rate:", round(acceptance_rate, 2), "%")
print("\nPost-selected results:")
print(counts_postselected)
```

```
Total shots: 1000
Accepted shots: 990
Rejected shots: 10
Acceptance rate: 99.0 %

Post-selected results:
{'0 101': 9, '0 100': 7, '0 110': 243, '0 001': 239, '0 011': 2, '0 111': 240, '0 010': 5, '0 000': 245}
```

```
from qiskit import transpile
from qiskit_aer import AerSimulator
import numpy as np

# Run ideal simulation
ideal_simulator = AerSimulator(method="stabilizer")

ideal_compiled = transpile(
    checked_circuit.circuit,
    ideal_simulator,
    optimization_level=0
)

ideal_result = ideal_simulator.run(
    ideal_compiled,
    shots=10000,
    seed_simulator=123
).result()

ideal_counts = ideal_result.get_counts()


# Extract the 3 data-qubit bits
def data_distribution(counts):
    distribution = {}

    for key, value in counts.items():
        data_bits = key.split()[-1]
        distribution[data_bits] = (
            distribution.get(data_bits, 0) + value
```

```
        )

    total = sum(distribution.values())

    return {
        k: v / total
        for k, v in distribution.items()
    }


ideal = data_distribution(ideal_counts)
noisy = data_distribution(counts)
filtered = data_distribution(counts_postselected)


# Classical distribution fidelity
def classical_fidelity(p, q):
    keys = set(p) | set(q)

    overlap = sum(
        np.sqrt(p.get(k, 0) * q.get(k, 0))
        for k in keys
    )

    return overlap ** 2


F_noisy = classical_fidelity(ideal, noisy)
F_filtered = classical_fidelity(ideal, filtered)

print("Ideal vs noisy fidelity:", round(F_noisy, 5))
print("Ideal vs post-selected fidelity:", round(F_filtered, 5))

print("\nImprovement:",
      round((F_filtered - F_noisy) * 100, 3),
      "percentage points")
```

```
Ideal vs noisy fidelity: 0.9759
Ideal vs post-selected fidelity: 0.97665

Improvement: 0.075 percentage points
```

```
noisy_fidelities = []
filtered_fidelities = []

for seed in range(30):

    result = simulator.run(
        compiled_circuit,
        shots=1000,
        seed_simulator=seed
    ).result()

    current_counts = result.get_counts()

    # Apply post-selection
    accepted_counts = {
        key: value
        for key, value in current_counts.items()
        if not ps_fn(key).any()
    }

    # Calculate distributions
    noisy_dist = data_distribution(current_counts)
    filtered_dist = data_distribution(accepted_counts)

    # Calculate fidelity
    noisy_fidelities.append(
        classical_fidelity(ideal, noisy_dist)
    )

    filtered_fidelities.append(
        classical_fidelity(ideal, filtered_dist)
    )
```

```
print("Average noisy fidelity:",
      np.mean(noisy_fidelities))

print("Average filtered fidelity:",
      np.mean(filtered_fidelities))

print("Average improvement:",
      np.mean(filtered_fidelities) -
      np.mean(noisy_fidelities))

print("Standard deviation (noisy):",
      np.std(noisy_fidelities, ddof=1))

print("Standard deviation (filtered):",
      np.std(filtered_fidelities, ddof=1))
```

```
Average noisy fidelity: 0.9758157222326943
Average filtered fidelity: 0.9765798061963473
Average improvement: 0.0007640839636530172
Standard deviation (noisy): 0.0008861546944824301
Standard deviation (filtered): 0.0008851566853840795
```

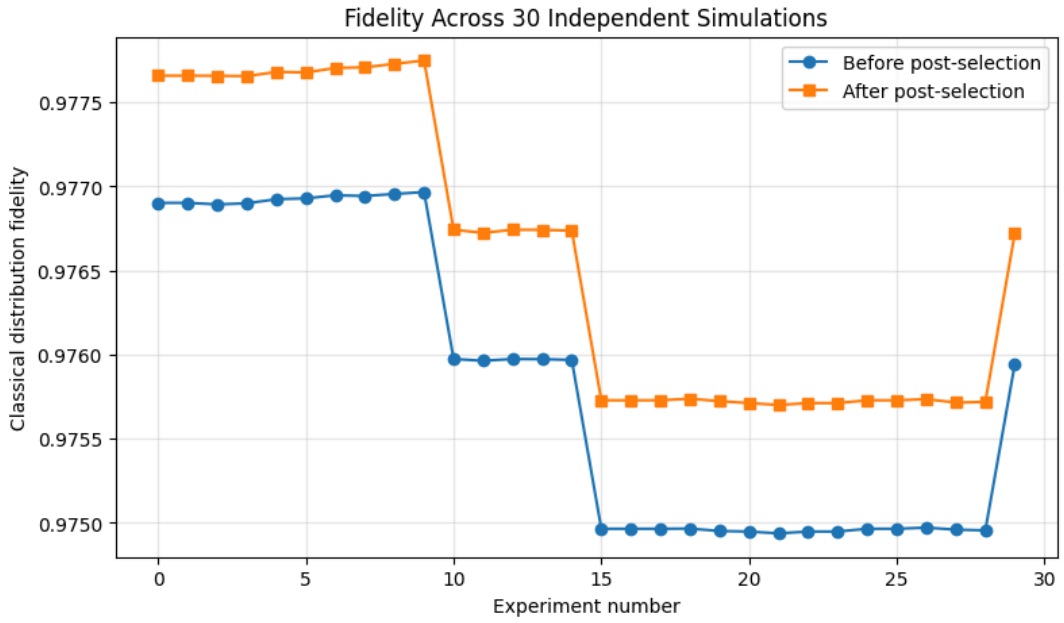
```
plt.figure(figsize=(9, 5))

plt.plot(
    noisy_fidelities,
    marker='o',
    label='Before post-selection'
)

plt.plot(
    filtered_fidelities,
    marker='s',
    label='After post-selection'
)

plt.xlabel("Experiment number")
plt.ylabel("Classical distribution fidelity")
plt.title("Fidelity Across 30 Independent Simulations")

plt.legend()
plt.grid(alpha=0.3)
plt.show()
```




```
import numpy as np

avg_before = np.mean(noisy_fidelities)
avg_after = np.mean(filtered_fidelities)

improvement = avg_after - avg_before
```

```
print("===== 30 SIMULATION RESULTS =====")
print(f"Average fidelity before post-selection: {avg_before:.6f}")
print(f"Average fidelity after post-selection:  {avg_after:.6f}")
print(f"Average improvement:                    {improvement:.6f}")
print(f"Improvement in percentage points:       {improvement * 100:.4f}")

print("\nPost-selection improved fidelity in",
      sum(a > b for a, b in zip(filtered_fidelities, noisy_fidelities)),
      "out of 30 simulations.")
```

```
===== 30 SIMULATION RESULTS =====
Average fidelity before post-selection: 0.975816
Average fidelity after post-selection:  0.976580
Average improvement:                    0.000764
Improvement in percentage points:       0.0764

Post-selection improved fidelity in 30 out of 30 simulations.
```

```
print("Experiment | Before | After | Difference")
print("-" * 50)

for i, (before, after) in enumerate(
    zip(noisy_fidelities, filtered_fidelities), start=1
):
    print(
        f"{i:10} | {before:.6f} | {after:.6f} | "
        f"{after - before:+.6f}"
    )
```

```
Experiment | Before | After | Difference
--------------------------------------------------
         1 | 0.976902 | 0.977658 | +0.000756
         2 | 0.976902 | 0.977658 | +0.000756
         3 | 0.976893 | 0.977657 | +0.000764
         4 | 0.976899 | 0.977655 | +0.000756
         5 | 0.976924 | 0.977680 | +0.000756
         6 | 0.976929 | 0.977677 | +0.000748
         7 | 0.976947 | 0.977704 | +0.000757
         8 | 0.976943 | 0.977708 | +0.000765
         9 | 0.976955 | 0.977728 | +0.000773
        10 | 0.976967 | 0.977748 | +0.000782
        11 | 0.975974 | 0.976742 | +0.000769
        12 | 0.975964 | 0.976724 | +0.000760
        13 | 0.975974 | 0.976742 | +0.000769
        14 | 0.975973 | 0.976742 | +0.000769
        15 | 0.975968 | 0.976737 | +0.000769
        16 | 0.974965 | 0.975729 | +0.000764
        17 | 0.974965 | 0.975729 | +0.000764
        18 | 0.974965 | 0.975729 | +0.000764
        19 | 0.974966 | 0.975738 | +0.000772
        20 | 0.974952 | 0.975723 | +0.000772
        21 | 0.974949 | 0.975712 | +0.000763
        22 | 0.974938 | 0.975701 | +0.000763
        23 | 0.974949 | 0.975712 | +0.000763
        24 | 0.974949 | 0.975712 | +0.000763
        25 | 0.974965 | 0.975729 | +0.000764
        26 | 0.974965 | 0.975729 | +0.000764
        27 | 0.974972 | 0.975735 | +0.000764
        28 | 0.974960 | 0.975715 | +0.000755
        29 | 0.974955 | 0.975719 | +0.000763
        30 | 0.975946 | 0.976722 | +0.000777
```

```
print("Number of before values:", len(noisy_fidelities))
print("Number of after values:", len(filtered_fidelities))

print("\nUnique before values:",
      len(set(noisy_fidelities)))

print("Unique after values:",
      len(set(filtered_fidelities)))

print("\nFirst 5 before values:", noisy_fidelities[:5])
print("First 5 after values:", filtered_fidelities[:5])

print("\nNoise model:", simulator.options.noise_model)
print("Circuit operations:", compiled_circuit.count_ops())
print("Number of circuit qubits:", compiled_circuit.num_qubits)
```

```
Number of before values: 30
Number of after values: 30

Unique before values: 22
Unique after values: 22

First 5 before values: [np.float64(0.976902031850407), np.float64(0.976902031850407), np.float64(0.9768930984993892), np.float
First 5 after values: [np.float64(0.9776579897952495), np.float64(0.9776579897952495), np.float64(0.9776571422664438), np.floa

Noise model: NoiseModel:
  Basis gates: ['cx', 'id', 'rz', 'sx']
  Instructions with noise: ['measure', 'cx']
  All-qubits errors: ['cx', 'measure']
Circuit operations: OrderedDict({'rz': 35, 'sx': 12, 'cx': 5, 'measure': 4})
Number of circuit qubits: 4
```

```
from collections import Counter

print("===== CIRCUIT CLASSICAL REGISTERS =====")
print(compiled_circuit.cregs)
print("Classical bits:", compiled_circuit.num_clbits)

print("\n===== MEASUREMENT MAPPING =====")
for instruction in compiled_circuit.data:
    if instruction.operation.name == "measure":
        print(
            "Qubit:", compiled_circuit.find_bit(instruction.qubits[0]).index,
            "-> Classical bit:",
            compiled_circuit.find_bit(instruction.clbits[0]).index
        )

print("\n===== POST-SELECTION CHECK =====")
print("Original shots:", sum(counts.values()))
print("Accepted shots:", sum(counts_postselected.values()))

print("\n===== RAW COUNT EXAMPLES =====")
for bitstring, number in list(counts.items())[:10]:
    print(repr(bitstring), ":", number)
```

```
===== CIRCUIT CLASSICAL REGISTERS =====
[ClassicalRegister(3, 'meas'), ClassicalRegister(1, 'checks_c')]
Classical bits: 4

===== MEASUREMENT MAPPING =====
Qubit: 0 -> Classical bit: 0
Qubit: 1 -> Classical bit: 1
Qubit: 2 -> Classical bit: 2
Qubit: 3 -> Classical bit: 3

===== POST-SELECTION CHECK =====
Original shots: 1000
Accepted shots: 990

===== RAW COUNT EXAMPLES =====
'1 001' : 1
'1 000' : 4
'0 101' : 9
'0 100' : 7
'1 101' : 1
'0 110' : 243
'1 111' : 4
'0 001' : 239
'0 011' : 2
'0 111' : 240
```

```
print("===== PAULICE CHECK DETAILS =====")
print("Target qubits:", checked_circuit.target_qubits)
print("Check ancilla qubits:", checked_circuit.check_qubits)
print("Check support:", checked_circuit.check_support)
print("Number of checks:", len(checked_circuit.check_qubits))

print("\n===== VERIFY SYNDROMES =====")
ps_fn = checked_circuit.get_postselection_method()

for bitstring in list(counts.keys())[:10]:
    syndrome = ps_fn(bitstring)
    print(
```

```
        f"Outcome: {bitstring:10s} "
        f"Syndrome: {syndrome} "
        f"Accepted: {not syndrome.any()}"
    )
```

```
===== PAULICE CHECK DETAILS =====
Target qubits: (1,)
Check ancilla qubits: (3,)
Check support: ((3,),)
Number of checks: 1

===== VERIFY SYNDROMES =====
Outcome: 1 001      Syndrome: [1] Accepted: False
Outcome: 1 000      Syndrome: [1] Accepted: False
Outcome: 0 101      Syndrome: [0] Accepted: True
Outcome: 0 100      Syndrome: [0] Accepted: True
Outcome: 1 101      Syndrome: [1] Accepted: False
Outcome: 0 110      Syndrome: [0] Accepted: True
Outcome: 1 111      Syndrome: [1] Accepted: False
Outcome: 0 001      Syndrome: [0] Accepted: True
Outcome: 0 011      Syndrome: [0] Accepted: True
Outcome: 0 111      Syndrome: [0] Accepted: True
```

```
print("===== ALL PAULICE VARIANTS =====")

for i, variant in enumerate(checked_variants):

    print(f"\nVariant {i}")
    print("Target qubits:", variant.target_qubits)
    print("Check ancillas:", variant.check_qubits)
    print("Check support:", variant.check_support)
    print("Number of checks:", len(variant.check_qubits))
    print("Cost:", variant.cost)

print("\n===== UNDETECTED PAULI ERRORS =====")
print(checked_circuit.uncovered_paulis)
```

```
===== ALL PAULICE VARIANTS =====

Variant 0
Target qubits: ()
Check ancillas: ()
Check support: ()
Number of checks: 0
Cost: 1.0245007045080268

Variant 1
Target qubits: (1,)
Check ancillas: (3,)
Check support: ((3,),)
Number of checks: 1
Cost: 1.024159033390146

===== UNDETECTED PAULI ERRORS =====
(UncoveredPauli(qubit=0, after_instruction=None, pauli='X'), UncoveredPauli(qubit=0, after_instruction=None, pauli='Y'), Uncov
```

```
print("===== UNCOVERED PAULI ERRORS =====")

uncovered = checked_circuit.uncovered_paulis

print("Total uncovered errors:", len(uncovered))

for error in uncovered:
    print(
        f"Qubit: {error.qubit}, "
        f"Pauli: {error.pauli}, "
        f"After instruction: {error.after_instruction}"
    )
```

```
===== UNCOVERED PAULI ERRORS =====
Total uncovered errors: 26
Qubit: 0, Pauli: X, After instruction: None
Qubit: 0, Pauli: Y, After instruction: None
Qubit: 0, Pauli: Z, After instruction: None
Qubit: 0, Pauli: X, After instruction: 7
Qubit: 0, Pauli: Y, After instruction: 7
Qubit: 0, Pauli: Z, After instruction: 7
Qubit: 1, Pauli: X, After instruction: None
```

```
Qubit: 1, Pauli: Y, After instruction: None
Qubit: 1, Pauli: Z, After instruction: None
Qubit: 1, Pauli: X, After instruction: 4
Qubit: 1, Pauli: X, After instruction: 7
Qubit: 1, Pauli: Y, After instruction: 17
Qubit: 1, Pauli: Z, After instruction: 22
Qubit: 1, Pauli: X, After instruction: 24
Qubit: 1, Pauli: Y, After instruction: 24
Qubit: 1, Pauli: Z, After instruction: 24
Qubit: 2, Pauli: X, After instruction: None
Qubit: 2, Pauli: Y, After instruction: None
Qubit: 2, Pauli: Z, After instruction: None
Qubit: 2, Pauli: X, After instruction: 22
Qubit: 2, Pauli: Y, After instruction: 22
Qubit: 2, Pauli: Z, After instruction: 22
Qubit: 3, Pauli: Z, After instruction: None
Qubit: 3, Pauli: Y, After instruction: 4
Qubit: 3, Pauli: X, After instruction: 17
Qubit: 3, Pauli: X, After instruction: 24
```

```
print("===== CIRCUIT INSTRUCTION MAPPING =====")

for i, instruction in enumerate(checked_circuit.circuit.data):

    operation = instruction.operation.name

    qubits = [
        checked_circuit.circuit.find_bit(q).index
        for q in instruction.qubits
    ]

    print(
        f"Instruction {i}: "
        f"Gate = {operation}, "
        f"Qubits = {qubits}"
    )
```

```
===== CIRCUIT INSTRUCTION MAPPING =====
Instruction 0: Gate = h, Qubits = [0]
Instruction 1: Gate = h, Qubits = [1]
Instruction 2: Gate = h, Qubits = [3]
Instruction 3: Gate = s, Qubits = [3]
Instruction 4: Gate = cx, Qubits = [3, 1]
Instruction 5: Gate = h, Qubits = [1]
Instruction 6: Gate = h, Qubits = [1]
Instruction 7: Gate = cx, Qubits = [0, 1]
Instruction 8: Gate = s, Qubits = [0]
Instruction 9: Gate = s, Qubits = [1]
Instruction 10: Gate = s, Qubits = [1]
Instruction 11: Gate = s, Qubits = [1]
Instruction 12: Gate = h, Qubits = [1]
Instruction 13: Gate = s, Qubits = [1]
Instruction 14: Gate = s, Qubits = [1]
Instruction 15: Gate = s, Qubits = [1]
Instruction 16: Gate = h, Qubits = [1]
Instruction 17: Gate = cx, Qubits = [3, 1]
Instruction 18: Gate = h, Qubits = [1]
Instruction 19: Gate = s, Qubits = [1]
Instruction 20: Gate = h, Qubits = [1]
Instruction 21: Gate = s, Qubits = [1]
Instruction 22: Gate = cx, Qubits = [1, 2]
Instruction 23: Gate = h, Qubits = [1]
Instruction 24: Gate = cx, Qubits = [3, 1]
Instruction 25: Gate = h, Qubits = [1]
Instruction 26: Gate = s, Qubits = [1]
Instruction 27: Gate = h, Qubits = [3]
Instruction 28: Gate = measure, Qubits = [0]
Instruction 29: Gate = measure, Qubits = [1]
Instruction 30: Gate = measure, Qubits = [2]
Instruction 31: Gate = measure, Qubits = [3]
```

```
from collections import Counter

uncovered = checked_circuit.uncovered_paulis

print("===== ERRORS BY QUBIT =====")

for qubit in range(4):
    count = sum(e.qubit == qubit for e in uncovered)
```

```
    print(f"Qubit {qubit}: {count}")

print("\n===== ERRORS BY PAULI TYPE =====")

for pauli in ["X", "Y", "Z"]:
    count = sum(e.pauli == pauli for e in uncovered)
    print(f"{pauli}: {count}")

print("\n===== ERRORS BY LOCATION =====")

locations = Counter(
    e.after_instruction for e in uncovered
)

for location, count in locations.items():
    print(f"After instruction {location}: {count}")
```

```
===== ERRORS BY QUBIT =====
Qubit 0: 6
Qubit 1: 10
Qubit 2: 6
Qubit 3: 4

===== ERRORS BY PAULI TYPE =====
X: 10
Y: 8
Z: 8

===== ERRORS BY LOCATION =====
After instruction None: 10
After instruction 7: 4
After instruction 4: 2
After instruction 17: 2
After instruction 22: 4
After instruction 24: 4
```

```
print("===== CIRCUIT INSTRUCTION MAPPING =====")

for i, instruction in enumerate(checked_circuit.circuit.data):

    operation = instruction.operation.name

    qubits = [
        checked_circuit.circuit.find_bit(q).index
        for q in instruction.qubits
    ]

    print(
        f"Instruction {i}: "
        f"Gate = {operation}, "
        f"Qubits = {qubits}"
    )
```

```
===== CIRCUIT INSTRUCTION MAPPING =====
Instruction 0: Gate = h, Qubits = [0]
Instruction 1: Gate = h, Qubits = [1]
Instruction 2: Gate = h, Qubits = [3]
Instruction 3: Gate = s, Qubits = [3]
Instruction 4: Gate = cx, Qubits = [3, 1]
Instruction 5: Gate = h, Qubits = [1]
Instruction 6: Gate = h, Qubits = [1]
Instruction 7: Gate = cx, Qubits = [0, 1]
Instruction 8: Gate = s, Qubits = [0]
Instruction 9: Gate = s, Qubits = [1]
Instruction 10: Gate = s, Qubits = [1]
Instruction 11: Gate = s, Qubits = [1]
Instruction 12: Gate = h, Qubits = [1]
Instruction 13: Gate = s, Qubits = [1]
Instruction 14: Gate = s, Qubits = [1]
Instruction 15: Gate = s, Qubits = [1]
Instruction 16: Gate = h, Qubits = [1]
Instruction 17: Gate = cx, Qubits = [3, 1]
Instruction 18: Gate = h, Qubits = [1]
Instruction 19: Gate = s, Qubits = [1]
Instruction 20: Gate = h, Qubits = [1]
Instruction 21: Gate = s, Qubits = [1]
Instruction 22: Gate = cx, Qubits = [1, 2]
Instruction 23: Gate = h, Qubits = [1]
Instruction 24: Gate = cx, Qubits = [3, 1]
```

```
Instruction 25: Gate = h, Qubits = [1]
Instruction 26: Gate = s, Qubits = [1]
Instruction 27: Gate = h, Qubits = [3]
Instruction 28: Gate = measure, Qubits = [0]
Instruction 29: Gate = measure, Qubits = [1]
Instruction 30: Gate = measure, Qubits = [2]
Instruction 31: Gate = measure, Qubits = [3]
```

```
print("===== COMPARISON OF PAULICE VARIANTS =====")

for i, variant in enumerate(checked_variants):

    print(f"\nVariant {i}")

    print("Target qubits:", variant.target_qubits)
    print("Check ancillas:", variant.check_qubits)
    print("Check support:", variant.check_support)

    if hasattr(variant, "uncovered_paulis"):

        errors = variant.uncovered_paulis

        print("Uncovered errors:", len(errors))

        for pauli in ["X", "Y", "Z"]:
            count = sum(
                e.pauli == pauli for e in errors
            )
            print(f"{pauli} errors:", count)

    else:
        print("Uncovered-error information unavailable")
```

```
===== COMPARISON OF PAULICE VARIANTS =====

Variant 0
Target qubits: ()
Check ancillas: ()
Check support: ()
Uncovered errors: 21
X errors: 7
Y errors: 7
Z errors: 7

Variant 1
Target qubits: (1,)
Check ancillas: (3,)
Check support: ((3,),)
Uncovered errors: 26
X errors: 10
Y errors: 8
Z errors: 8
```

```
print("===== FAIR DATA-QUBIT COMPARISON =====")

for i, variant in enumerate(checked_variants):

    errors = variant.uncovered_paulis

    data_errors = [
        e for e in errors
        if e.qubit in [0, 1, 2]
    ]

    ancilla_errors = [
        e for e in errors
        if e.qubit == 3
    ]

    print(f"\nVariant {i}")
    print("Data-qubit uncovered errors:", len(data_errors))
    print("Ancilla uncovered errors:", len(ancilla_errors))

    for pauli in ["X", "Y", "Z"]:
        count = sum(e.pauli == pauli for e in data_errors)
        print(f"Data {pauli} errors:", count)
```

```
===== FAIR DATA-QUBIT COMPARISON =====

Variant 0
Data-qubit uncovered errors: 21
Ancilla uncovered errors: 0
Data X errors: 7
Data Y errors: 7
Data Z errors: 7

Variant 1
Data-qubit uncovered errors: 22
Ancilla uncovered errors: 4
Data X errors: 8
Data Y errors: 7
Data Z errors: 7
```

```
print("===== ERROR COVERAGE COMPARISON =====")

v0_errors = {
    (e.qubit, e.after_instruction, e.pauli)
    for e in checked_variants[0].uncovered_paulis
    if e.qubit in [0, 1, 2]
}

v1_errors = {
    (e.qubit, e.after_instruction, e.pauli)
    for e in checked_variants[1].uncovered_paulis
    if e.qubit in [0, 1, 2]
}

newly_uncovered = v1_errors - v0_errors
no_longer_uncovered = v0_errors - v1_errors

print("Variant 0 uncovered:", len(v0_errors))
print("Variant 1 uncovered:", len(v1_errors))

print("\nNewly uncovered in Variant 1:")
for error in sorted(newly_uncovered, key=str):
    print(error)

print("\nNo longer uncovered in Variant 1:")
for error in sorted(no_longer_uncovered, key=str):
    print(error)
```

```
===== ERROR COVERAGE COMPARISON =====
Variant 0 uncovered: 21
Variant 1 uncovered: 22

Newly uncovered in Variant 1:
(0, 7, 'X')
(0, 7, 'Y')
(0, 7, 'Z')
(1, 17, 'Y')
(1, 22, 'Z')
(1, 24, 'X')
(1, 24, 'Y')
(1, 24, 'Z')
(1, 7, 'X')
(2, 22, 'X')
(2, 22, 'Y')
(2, 22, 'Z')

No longer uncovered in Variant 1:
(0, 2, 'X')
(0, 2, 'Y')
(0, 2, 'Z')
(1, 2, 'X')
(1, 2, 'Y')
(1, 2, 'Z')
(1, 4, 'Y')
(1, 4, 'Z')
(2, 4, 'X')
(2, 4, 'Y')
(2, 4, 'Z')
```

```
print("===== VARIANT COVERAGE SUMMARY =====")

for i, variant in enumerate(checked_variants):
```

```
    data_errors = {
        (e.qubit, e.after_instruction, e.pauli)
        for e in variant.uncovered_paulis
        if e.qubit in [0, 1, 2]
    }

    print(f"\nVariant {i}")
    print("Target qubits:", variant.target_qubits)
    print("Check qubits:", variant.check_qubits)
    print("Uncovered data errors:", len(data_errors))
```

```
===== VARIANT COVERAGE SUMMARY =====

Variant 0
Target qubits: ()
Check qubits: ()
Uncovered data errors: 21

Variant 1
Target qubits: (1,)
Check qubits: (3,)
Uncovered data errors: 22
```

```
print("Number of generated variants:", len(checked_variants))

print("\nOriginal circuit qubits:", qc.num_qubits)
print("T-doped circuit qubits:", tqc.num_qubits)
print("Clifford QEC circuit qubits:", clifford_qec.num_qubits)
```

```
Number of generated variants: 2

Original circuit qubits: 3
T-doped circuit qubits: 3
Clifford QEC circuit qubits: 3
```

```
import inspect

print(inspect.signature(add_pauli_checks))
print("\nFunction documentation:")
print(inspect.getdoc(add_pauli_checks))
```

```
        from dataclasses import replace
        from qiskit import ClassicalRegister

        payload = circuit.remove_final_measurements(inplace=False)
        noise = NoiseModel(gate_noise=1e-3)
        checked = add_pauli_checks(payload, [1], noise, stabilizers="all")[-1]

        measured = checked.circuit.copy()
        measured.h(range(3))  # any basis rotation of the payload
        meas = ClassicalRegister(3, "meas")
        measured.add_register(meas)
        measured.measure(range(3), meas)
        checked = replace(checked, circuit=measured)
        accept = checked.get_postselection_method()  # takes a bitstring of ``measured``

References:
    S. Martiel and A. Javadi-Abhari, *Low-overhead error detection with spacetime codes*,
    `arXiv:2504.15725 <https://arxiv.org/abs/2504.15725>`_. Supplementary Sections II to VI
    describe how effective checks are found.
```

```
from qiskit_paulice import NoiseModel, add_pauli_checks

search_noise = NoiseModel(
    gate_noise=0.001,
    readout_noise=0.01
)

genetic_variants = add_pauli_checks(
    circuit=clifford_qec,
    target_qubits=[0, 1, 2],
    noise_model=search_noise,
    cost="gamma",
    method="genetic",
    seed=42
)

print("Number of genetic variants:", len(genetic_variants))

for i, variant in enumerate(genetic_variants):
    data_errors = {
        (e.qubit, e.after_instruction, e.pauli)
        for e in variant.uncovered_paulis
        if e.qubit in [0, 1, 2]
    }

    print(f"\nVariant {i}")
    print("Target qubits:", variant.target_qubits)
    print("Check qubits:", variant.check_qubits)
    print("Uncovered data errors:", len(data_errors))
```

```
Number of genetic variants: 4

Variant 0
Target qubits: ()
Check qubits: ()
Uncovered data errors: 21

Variant 1
Target qubits: (0,)
Check qubits: (3,)
Uncovered data errors: 20

Variant 2
Target qubits: (0, 1)
Check qubits: (3, 4)
Uncovered data errors: 17

Variant 3
Target qubits: (0, 1, 2)
Check qubits: (3, 4, 5)
Uncovered data errors: 16
```

```
print("===== GENETIC VARIANT COST COMPARISON =====")

for i, variant in enumerate(genetic_variants):
    print(f"\nVariant {i}")
    print("Cost:", variant.cost)
```

```
    print("Target qubits:", variant.target_qubits)
    print("Check qubits:", variant.check_qubits)
```

```
===== GENETIC VARIANT COST COMPARISON =====

Variant 0
Cost: 1.0245007045080268
Target qubits: ()
Check qubits: ()

Variant 1
Cost: 1.0238174762196273
Target qubits: (0,)
Check qubits: (3,)

Variant 2
Cost: 1.0224523862513344
Target qubits: (0, 1)
Check qubits: (3, 4)

Variant 3
Cost: 1.0020033385263076
Target qubits: (0, 1, 2)
Check qubits: (3, 4, 5)
```

```
best_variant = genetic_variants[-1]

print("===== BEST GENETIC VARIANT =====")
print("Target qubits:", best_variant.target_qubits)
print("Check qubits:", best_variant.check_qubits)
print("Check support:", best_variant.check_support)
print("Number of checks:", len(best_variant.check_qubits))

print("\nUncovered Pauli errors:")
for error in best_variant.uncovered_paulis:
    print(error.qubit, error.pauli, error.after_instruction)
```

```
===== BEST GENETIC VARIANT =====
Target qubits: (0, 1, 2)
Check qubits: (3, 4, 5)
Check support: ((0, 3), (0, 1, 4), (1, 2, 5))
Number of checks: 3

Uncovered Pauli errors:
0 X None
0 Y None
0 Z None
0 X 6
0 Z 16
1 X None
1 Y None
1 Z None
1 X 12
1 Z 16
1 Z 22
2 X None
2 Y None
2 Z None
2 X 20
2 Z 22
3 Z None
3 X 6
4 Z None
4 X 12
5 Z None
5 X 20
```

```
print("===== GENETIC VARIANT 3 CIRCUIT =====")

print(best_variant.circuit.draw(output="text"))

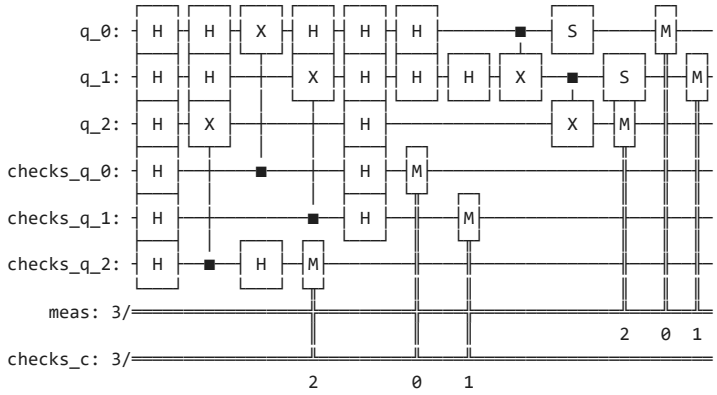
print("\nNumber of qubits:", best_variant.circuit.num_qubits)
print("Number of classical bits:", best_variant.circuit.num_clbits)

print("\nClassical registers:")
for register in best_variant.circuit.cregs:
    print(register.name, ":", register.size)

print("\nPost-selection method:")
genetic_ps_fn = best_variant.get_postselection_method()
```

```
print(genetic_ps_fn)
```

```
===== GENETIC VARIANT 3 CIRCUIT =====
```




```
Number of qubits: 6
Number of classical bits: 6

Classical registers:
meas : 3
checks_c : 3

Post-selection method:
<function CheckedCircuit.get_postselection_method.<locals>._aux at 0x7da51f200d60>
```

```
from qiskit_aer import AerSimulator
from qiskit_aer.noise import NoiseModel as AerNoiseModel
from qiskit_aer.noise import depolarizing_error, ReadoutError

# Create Aer noise model
aer_noise = AerNoiseModel()

# Add depolarizing noise to CX gates
cx_error = depolarizing_error(0.001, 2)
aer_noise.add_all_qubit_quantum_error(cx_error, ["cx"])

# Add readout noise
readout_error = ReadoutError([
    [0.99, 0.01],
    [0.01, 0.99]
])

aer_noise.add_all_qubit_readout_error(readout_error)

# Create simulator
simulator = AerSimulator(
    method="stabilizer",
    noise_model=aer_noise
)

print("Noisy simulator created successfully!")
```

```
Noisy simulator created successfully!
```

```
# Get the circuit from Genetic Variant 3
genetic_circuit = best_variant.circuit

# Run simulation
genetic_job = simulator.run(
    genetic_circuit,
    shots=1000
)

genetic_result = genetic_job.result()
genetic_counts = genetic_result.get_counts()

print("===== GENETIC VARIANT 3 RESULTS =====")
print("Total shots:", sum(genetic_counts.values()))
print("\nMeasurement counts:")
```

```
for bitstring, count in genetic_counts.items():
    print(bitstring, ":", count)
```

```
===== GENETIC VARIANT 3 RESULTS =====
Total shots: 1000

Measurement counts:
000 111 : 2
000 100 : 1
100 000 : 2
001 011 : 3
011 101 : 2
010 100 : 1
110 110 : 2
011 111 : 2
011 000 : 3
010 110 : 223
001 111 : 226
011 001 : 235
000 000 : 257
000 010 : 3
110 010 : 1
000 001 : 4
010 010 : 5
010 111 : 4
001 110 : 1
010 001 : 4
111 001 : 2
011 011 : 3
010 000 : 3
101 111 : 3
000 110 : 5
001 000 : 2
011 110 : 1
```

```
print("Sample measurement bitstrings:")
for bitstring in list(genetic_counts.keys())[:5]:
    print(repr(bitstring))

print("\nPost-selection function:")
print(genetic_ps_fn)
```

```
Sample measurement bitstrings:
'000 111'
'000 100'
'100 000'
'001 011'
'011 101'

Post-selection function:
<function CheckedCircuit.get_postselection_method.<locals>._aux at 0x7da51f200d60>
```

```
print("===== TESTING POST-SELECTION =====")

for bitstring in list(genetic_counts.keys())[:5]:
    print("\nBitstring:", repr(bitstring))

    try:
        result = genetic_ps_fn(bitstring)
        print("Post-selection result:", result)
        print("Result type:", type(result))

    except Exception as error:
        print("Error:", repr(error))
```

```
===== TESTING POST-SELECTION =====

Bitstring: '000 111'
Post-selection result: [1 0 0]
Result type: <class 'numpy.ndarray'>

Bitstring: '000 100'
Post-selection result: [0 0 1]
Result type: <class 'numpy.ndarray'>

Bitstring: '100 000'
Post-selection result: [0 0 1]
Result type: <class 'numpy.ndarray'>

Bitstring: '001 011'
```

```
Post-selection result: [0 0 1]
Result type: <class 'numpy.ndarray'>

Bitstring: '011 101'
Post-selection result: [0 0 1]
Result type: <class 'numpy.ndarray'>
```

```
import numpy as np

# Apply post-selection
genetic_postselected_counts = {}

for bitstring, count in genetic_counts.items():

    syndrome = genetic_ps_fn(bitstring)

    # Accept only when all syndrome bits are zero
    if np.all(syndrome == 0):
        genetic_postselected_counts[bitstring] = count

total_shots = sum(genetic_counts.values())
accepted_shots = sum(genetic_postselected_counts.values())
rejected_shots = total_shots - accepted_shots

acceptance_rate = (accepted_shots / total_shots) * 100

print("===== GENETIC VARIANT 3 POST-SELECTION =====")
print("Total shots:", total_shots)
print("Accepted shots:", accepted_shots)
print("Rejected shots:", rejected_shots)
print(f"Acceptance rate: {acceptance_rate:.2f}%")
```

```
===== GENETIC VARIANT 3 POST-SELECTION =====
Total shots: 1000
Accepted shots: 942
Rejected shots: 58
Acceptance rate: 94.20%
```

```
from qiskit.quantum_info import state_fidelity
from qiskit.result import marginal_counts

# Extract only the 3 data-qubit measurement bits
genetic_data_counts = marginal_counts(
    genetic_counts,
    indices=[0, 1, 2]
)

genetic_postselected_data_counts = marginal_counts(
    genetic_postselected_counts,
    indices=[0, 1, 2]
)

# Ideal measurement distribution
ideal_counts = simulator.run(
    clifford_qec,
    shots=10000
).result().get_counts()

ideal_data_counts = marginal_counts(
    ideal_counts,
    indices=[0, 1, 2]
)

print("Noisy data counts:")
print(genetic_data_counts)

print("\nPostselected data counts:")
print(genetic_postselected_data_counts)

print("\nIdeal data counts:")
print(ideal_data_counts)
```

```
Noisy data counts:
{'111': 237, '100': 2, '000': 267, '011': 6, '101': 2, '110': 232, '001': 245, '010': 9}

Postselected data counts:
```

```
{'110': 223, '111': 226, '001': 235, '000': 257, '010': 1}

Ideal data counts:
{'101': 50, '100': 53, '010': 47, '000': 2452, '011': 58, '111': 2453, '110': 2441, '001': 2446}
```

```
ideal_counts = simulator.run(
    clifford_qec,
    shots=10000
).result().get_counts()
```

```
# Create a simulator WITHOUT a noise model
ideal_simulator = AerSimulator(method="stabilizer")

# Run the original Clifford QEC circuit without noise
true_ideal_counts = ideal_simulator.run(
    clifford_qec,
    shots=10000
).result().get_counts()

# Extract only the three data-qubit bits
true_ideal_data_counts = marginal_counts(
    true_ideal_counts,
    indices=[0, 1, 2]
)

print("===== TRUE IDEAL DISTRIBUTION =====")
print(true_ideal_data_counts)

print("\nTotal ideal shots:",
      sum(true_ideal_data_counts.values()))
```

```
===== TRUE IDEAL DISTRIBUTION =====
{'000': 2502, '111': 2420, '001': 2585, '110': 2493}

Total ideal shots: 10000
```

```
import numpy as np

def calculate_classical_fidelity(counts1, counts2):

    # Use the same measurement outcomes
    all_keys = set(counts1.keys()) | set(counts2.keys())

    total1 = sum(counts1.values())
    total2 = sum(counts2.values())

    fidelity_sum = 0

    for key in all_keys:
        p = counts1.get(key, 0) / total1
        q = counts2.get(key, 0) / total2

        fidelity_sum += np.sqrt(p * q)

    return fidelity_sum ** 2


# Fidelity before post-selection
fidelity_before = calculate_classical_fidelity(
    genetic_data_counts,
    true_ideal_data_counts
)

# Fidelity after post-selection
fidelity_after = calculate_classical_fidelity(
    genetic_postselected_data_counts,
    true_ideal_data_counts
)

improvement = fidelity_after - fidelity_before

print("===== GENETIC VARIANT 3 FIDELITY =====")

print(f"Fidelity before post-selection: {fidelity_before:.6f}")
print(f"Fidelity after post-selection:  {fidelity_after:.6f}")
```

```
print(f"\nAbsolute improvement: {improvement:.6f}")
print(f"Percentage-point improvement: {improvement * 100:.4f}")
```

```
===== GENETIC VARIANT 3 FIDELITY =====
Fidelity before post-selection: 0.980307
Fidelity after post-selection:  0.998202

Absolute improvement: 0.017895
Percentage-point improvement: 1.7895
```

```
import pandas as pd

comparison = pd.DataFrame({
    "Parameter": [
        "Number of Pauli checks",
        "Uncovered data errors",
        "Total shots",
        "Accepted shots",
        "Rejected shots",
        "Acceptance rate (%)",
        "Fidelity before post-selection",
        "Fidelity after post-selection",
        "Fidelity improvement (percentage points)"
    ],

    "Windowed Method": [
        1,
        22,
        1000,
        990,
        10,
        99.00,
        0.9759,
        0.97665,
        0.075
    ],

    "Genetic Method": [
        3,
        16,
        1000,
        942,
        58,
        94.20,
        0.980307,
        0.998202,
        1.7895
    ]
})

print("===== WINDOWED VS GENETIC COMPARISON =====")
display(comparison)
```

```
===== WINDOWED VS GENETIC COMPARISON =====
```

	Parameter	Windowed Method	Genetic Method
0	Number of Pauli checks	1.00000	3.000000
1	Uncovered data errors	22.00000	16.000000
2	Total shots	1000.00000	1000.000000
3	Accepted shots	990.00000	942.000000
4	Rejected shots	10.00000	58.000000
5	Acceptance rate (%)	99.00000	94.200000
6	Fidelity before post-selection	0.97590	0.980307
7	Fidelity after post-selection	0.97665	0.998202
8	Fidelity improvement (percentage points)	0.07500	1.789500

Next steps: Generate code with comparison | New interactive sheet

```
import numpy as np
```

```
import pandas as pd
from qiskit.result import marginal_counts

# Select the two circuits
windowed_circuit = checked_variants[1].circuit
genetic_circuit = genetic_variants[3].circuit

# Get their post-selection functions
windowed_ps = checked_variants[1].get_postselection_method()
genetic_ps = genetic_variants[3].get_postselection_method()


# Classical measurement-distribution fidelity
def calculate_fidelity(counts1, counts2):

    keys = set(counts1) | set(counts2)

    total1 = sum(counts1.values())
    total2 = sum(counts2.values())

    value = 0

    for key in keys:
        p = counts1.get(key, 0) / total1
        q = counts2.get(key, 0) / total2
        value += np.sqrt(p * q)

    return value ** 2


# Run one experiment
def run_experiment(circuit, postselection, seed):

    result = simulator.run(
        circuit,
        shots=1000,
        seed_simulator=seed
    ).result()

    counts = result.get_counts()

    accepted = {
        bitstring: count
        for bitstring, count in counts.items()
        if np.all(postselection(bitstring) == 0)
    }

    data_counts = marginal_counts(counts, indices=[0, 1, 2])
    accepted_data = marginal_counts(accepted, indices=[0, 1, 2])

    acceptance = sum(accepted.values()) / 1000

    before = calculate_fidelity(
        data_counts,
        true_ideal_data_counts
    )

    after = calculate_fidelity(
        accepted_data,
        true_ideal_data_counts
    )

    return acceptance * 100, before, after


# Run 20 experiments for both methods
results = []

for seed in range(20):

    w_accept, w_before, w_after = run_experiment(
        windowed_circuit, windowed_ps, seed
    )

    g_accept, g_before, g_after = run_experiment(
        genetic_circuit, genetic_ps, seed
    )
```

```
    }

    results.append({
        "Seed": seed,
        "Windowed Acceptance (%)": w_accept,
        "Windowed Fidelity Before": w_before,
        "Windowed Fidelity After": w_after,
        "Genetic Acceptance (%)": g_accept,
        "Genetic Fidelity Before": g_before,
        "Genetic Fidelity After": g_after
    })

results_df = pd.DataFrame(results)

print("===== 20 EXPERIMENT RESULTS =====")
display(results_df.round(6))
```

```
===== 20 EXPERIMENT RESULTS =====
```

	Seed	Windowed Acceptance (%)	Windowed Fidelity Before	Windowed Fidelity After	Genetic Acceptance (%)	Genetic Fidelity Before	Genetic Fidelity After
0	0	99.1	0.976856	0.977715	92.5	0.976867	0.997660
1	1	99.1	0.976856	0.977715	92.5	0.976867	0.997660
2	2	99.1	0.976829	0.977696	92.5	0.976840	0.997638
3	3	99.1	0.976834	0.977692	92.6	0.976842	0.997630
4	4	99.1	0.976823	0.977682	92.6	0.976834	0.997612
5	5	99.1	0.976846	0.977696	92.6	0.976857	0.997633
6	6	99.1	0.976831	0.977690	92.6	0.976846	0.997624
7	7	99.1	0.976829	0.977696	92.6	0.976846	0.997635
8	8	99.1	0.976808	0.977683	92.7	0.976829	0.997617
9	9	99.1	0.976767	0.977650	92.7	0.976790	0.997569
10	10	99.1	0.975769	0.976639	92.6	0.975792	0.997563
11	11	99.1	0.975792	0.976654	92.6	0.975811	0.997579
12	12	99.1	0.975769	0.976639	92.6	0.975792	0.997563
13	13	99.1	0.975749	0.976618	92.6	0.975770	0.997529
14	14	99.1	0.975724	0.976594	92.7	0.975743	0.997522
15	15	99.1	0.974698	0.975563	92.6	0.974717	0.997490
16	16	99.1	0.974698	0.975563	92.6	0.974717	0.997490
17	17	99.1	0.974698	0.975563	92.6	0.974717	0.997490
18	18	99.1	0.974647	0.975519	92.6	0.974668	0.997433
19	19	99.1	0.974667	0.975540	92.6	0.974686	0.997463

```
import pandas as pd
import numpy as np

# Calculate fidelity improvement for each experiment
results_df["Windowed Improvement"] = (
    results_df["Windowed Fidelity After"]
    - results_df["Windowed Fidelity Before"]
) * 100

results_df["Genetic Improvement"] = (
    results_df["Genetic Fidelity After"]
    - results_df["Genetic Fidelity Before"]
) * 100

# Calculate summary statistics
summary = pd.DataFrame({
    "Metric": [
        "Acceptance Rate (%)",
        "Fidelity Before",
        "Fidelity After",
        "Fidelity Improvement (percentage points)"
```

```
    ],

    "Windowed Mean": [
        results_df["Windowed Acceptance (%)"].mean(),
        results_df["Windowed Fidelity Before"].mean(),
        results_df["Windowed Fidelity After"].mean(),
        results_df["Windowed Improvement"].mean()
    ],

    "Windowed Std Dev": [
        results_df["Windowed Acceptance (%)"].std(),
        results_df["Windowed Fidelity Before"].std(),
        results_df["Windowed Fidelity After"].std(),
        results_df["Windowed Improvement"].std()
    ],

    "Genetic Mean": [
        results_df["Genetic Acceptance (%)"].mean(),
        results_df["Genetic Fidelity Before"].mean(),
        results_df["Genetic Fidelity After"].mean(),
        results_df["Genetic Improvement"].mean()
    ],

    "Genetic Std Dev": [
        results_df["Genetic Acceptance (%)"].std(),
        results_df["Genetic Fidelity Before"].std(),
        results_df["Genetic Fidelity After"].std(),
        results_df["Genetic Improvement"].std()
    ]
})

print("===== FINAL STATISTICAL SUMMARY =====")
display(summary.round(6))
```

```
===== FINAL STATISTICAL SUMMARY =====
```

	Metric	Windowed Mean	Windowed Std Dev	Genetic Mean	Genetic Std Dev
0	Acceptance Rate (%)	99.100000	0.000000	92.600000	0.056195
1	Fidelity Before	0.976025	0.000913	0.976041	0.000910
2	Fidelity After	0.976890	0.000911	0.997570	0.000070
3	Fidelity Improvement (percentage points)	0.086554	0.000732	2.152844	0.084408

```
print("===== UNIQUE RESULT CHECK =====")

windowed_cols = [
    "Windowed Acceptance (%)",
    "Windowed Fidelity Before",
    "Windowed Fidelity After"
]

genetic_cols = [
    "Genetic Acceptance (%)",
    "Genetic Fidelity Before",
    "Genetic Fidelity After"
]

print("Unique Windowed results:",
      results_df[windowed_cols].drop_duplicates().shape[0])

print("Unique Genetic results:",
      results_df[genetic_cols].drop_duplicates().shape[0])

print("\nWindowed results:")
display(results_df[windowed_cols])

print("\nGenetic results:")
display(results_df[genetic_cols])
```

```
windowed_raw_results = []
genetic_raw_results = []

for seed in range(20):

    # Windowed raw counts
    w_result = simulator.run(
        windowed_circuit,
        shots=1000,
        seed_simulator=seed
    ).result()

    w_counts = w_result.get_counts()
    windowed_raw_results.append(w_counts)

    # Genetic raw counts
    g_result = simulator.run(
        genetic_circuit,
        shots=1000,
        seed_simulator=seed
    ).result()

    g_counts = g_result.get_counts()
    genetic_raw_results.append(g_counts)


print("===== RAW COUNT UNIQUENESS =====")

print(
    "Unique Windowed raw count sets:",
    len({str(sorted(c.items())) for c in windowed_raw_results})
)

print(
    "Unique Genetic raw count sets:",
    len({str(sorted(c.items())) for c in genetic_raw_results})
)
```

19	99.1	0.974667	0.975540

```
===== RAW COUNT UNIQUENESS =====
Unique Windowed raw count sets: 16
Unique Genetic raw count sets: 16
```

Genetic Acceptance (%)	Genetic Fidelity Before	Genetic Fidelity After

```
# ========================================
# 10,000-SHOT EXPERIMENT (20 SEEDS)
# ========================================

results_10000 = []

SHOTS = 10000

for seed in range(20):

    # Windowed method
    w_result = simulator.run(
        windowed_circuit,
        shots=SHOTS,
        seed_simulator=seed
    ).result()

    w_counts = w_result.get_counts()

    w_accepted = {
        bitstring: count
        for bitstring, count in w_counts.items()
        if np.all(windowed_ps(bitstring) == 0)
    }

    w_data = marginal_counts(w_counts, indices=[0, 1, 2])
    w_post_data = marginal_counts(w_accepted, indices=[0, 1, 2])

    w_acceptance = sum(w_accepted.values()) / SHOTS * 100

    w_before = calculate_fidelity(
        w_data, true_ideal_data_counts
    )
```

```
    w_after = calculate_fidelity(
        w_post_data, true_ideal_data_counts
    )

    # Genetic method
    g_result = simulator.run(
        genetic_circuit,
        shots=SHOTS,
        seed_simulator=seed
    ).result()

    g_counts = g_result.get_counts()

    g_accepted = {
        bitstring: count
        for bitstring, count in g_counts.items()
        if np.all(genetic_ps(bitstring) == 0)
    }

    g_data = marginal_counts(g_counts, indices=[0, 1, 2])
    g_post_data = marginal_counts(g_accepted, indices=[0, 1, 2])

    g_acceptance = sum(g_accepted.values()) / SHOTS * 100

    g_before = calculate_fidelity(
        g_data, true_ideal_data_counts
    )

    g_after = calculate_fidelity(
        g_post_data, true_ideal_data_counts
    )

    results_10000.append({
        "Seed": seed,
        "Windowed Acceptance (%)": w_acceptance,
        "Windowed Fidelity Before": w_before,
        "Windowed Fidelity After": w_after,
        "Genetic Acceptance (%)": g_acceptance,
        "Genetic Fidelity Before": g_before,
        "Genetic Fidelity After": g_after
    })

results_10000_df = pd.DataFrame(results_10000)

print("10,000-shot experiment completed!")

display(results_10000_df.round(6))
```

```
10,000-shot experiment completed!
```

	Seed	Windowed Acceptance (%)	Windowed Fidelity Before	Windowed Fidelity After	Genetic Acceptance (%)	Genetic Fidelity Before	Genetic Fidelity After
0	0	98.7	0.978807	0.979136	93.82	0.978905	0.999358
1	1	98.7	0.978808	0.979137	93.82	0.978907	0.999360
2	2	98.7	0.978808	0.979137	93.82	0.978907	0.999360
3	3	98.7	0.978809	0.979138	93.83	0.978908	0.999360
4	4	98.7	0.978808	0.979137	93.83	0.978906	0.999359
5	5	98.7	0.978808	0.979137	93.83	0.978907	0.999359
6	6	98.7	0.978808	0.979137	93.83	0.978906	0.999359
7	7	98.7	0.978809	0.979138	93.83	0.978907	0.999359
8	8	98.7	0.978809	0.979138	93.84	0.978908	0.999361
9	9	98.7	0.978807	0.979136	93.84	0.978906	0.999358
10	10	98.7	0.978808	0.979137	93.84	0.978907	0.999359
11	11	98.7	0.978810	0.979139	93.84	0.978909	0.999361
12	12	98.7	0.978808	0.979137	93.84	0.978907	0.999359
13	13	98.7	0.978807	0.979136	93.84	0.978906	0.999358
14	14	98.7	0.978804	0.979133	93.85	0.978903	0.999356
15	15	98.7	0.978803	0.979132	93.85	0.978902	0.999355
16	16	98.7	0.978805	0.979134	93.84	0.978904	0.999355
17	17	98.7	0.978806	0.979135	93.84	0.978905	0.999357
18	18	98.7	0.978804	0.979133	93.84	0.978903	0.999355
19	19	98.7	0.978804	0.979133	93.84	0.978903	0.999355

```
# Calculate fidelity improvement in percentage points

results_10000_df["Windowed Improvement (pp)"] = (
    results_10000_df["Windowed Fidelity After"]
    - results_10000_df["Windowed Fidelity Before"]
) * 100

results_10000_df["Genetic Improvement (pp)"] = (
    results_10000_df["Genetic Fidelity After"]
    - results_10000_df["Genetic Fidelity Before"]
) * 100


# Prepare statistical summary

summary_10000 = pd.DataFrame({
    "Metric": [
        "Acceptance Rate (%)",
        "Fidelity Before",
        "Fidelity After",
        "Fidelity Improvement (percentage points)"
    ],

    "Windowed Mean": [
        results_10000_df["Windowed Acceptance (%)"].mean(),
        results_10000_df["Windowed Fidelity Before"].mean(),
        results_10000_df["Windowed Fidelity After"].mean(),
        results_10000_df["Windowed Improvement (pp)"].mean()
    ],

    "Windowed Std Dev": [
        results_10000_df["Windowed Acceptance (%)"].std(),
        results_10000_df["Windowed Fidelity Before"].std(),
        results_10000_df["Windowed Fidelity After"].std(),
        results_10000_df["Windowed Improvement (pp)"].std()
    ],
```

```
    "Genetic Mean": [
        results_10000_df["Genetic Acceptance (%)"].mean(),
        results_10000_df["Genetic Fidelity Before"].mean(),
        results_10000_df["Genetic Fidelity After"].mean(),
        results_10000_df["Genetic Improvement (pp)"].mean()
    ],

    "Genetic Std Dev": [
        results_10000_df["Genetic Acceptance (%)"].std(),
        results_10000_df["Genetic Fidelity Before"].std(),
        results_10000_df["Genetic Fidelity After"].std(),
        results_10000_df["Genetic Improvement (pp)"].std()
    ]
})

print("===== FINAL 10,000-SHOT STATISTICAL SUMMARY =====")
display(summary_10000.round(8))
```

```
===== FINAL 10,000-SHOT STATISTICAL SUMMARY =====
```

	Metric	Windowed Mean	Windowed Std Dev	Genetic Mean	Genetic Std Dev
0	Acceptance Rate (%)	98.700000	0.000000	93.835500	0.008870
1	Fidelity Before	0.978807	0.000002	0.978906	0.000002
2	Fidelity After	0.979136	0.000002	0.999358	0.000002
3	Fidelity Improvement (percentage points)	0.032898	0.000007	2.045235	0.000041

```
import matplotlib.pyplot as plt
import numpy as np

methods = ["Windowed", "Genetic"]

fidelity_before = [
    summary_10000.loc[1, "Windowed Mean"],
    summary_10000.loc[1, "Genetic Mean"]
]

fidelity_after = [
    summary_10000.loc[2, "Windowed Mean"],
    summary_10000.loc[2, "Genetic Mean"]
]

x = np.arange(len(methods))
width = 0.35

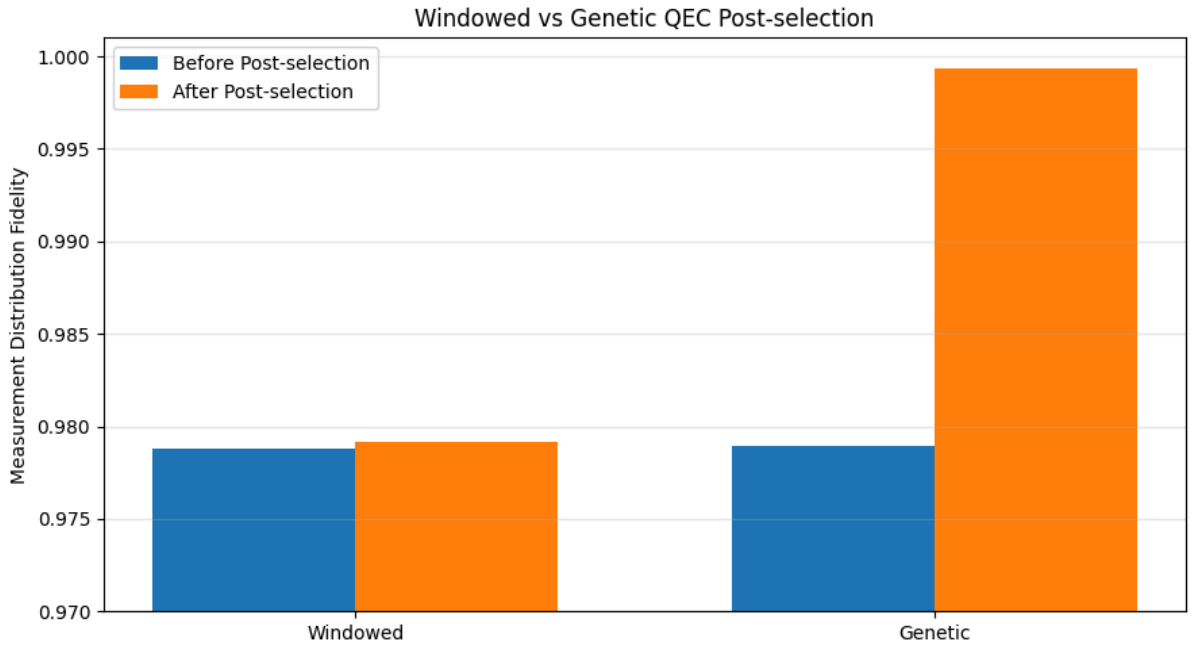
plt.figure(figsize=(9, 5))

plt.bar(
    x - width/2,
    fidelity_before,
    width,
    label="Before Post-selection"
)

plt.bar(
    x + width/2,
    fidelity_after,
    width,
    label="After Post-selection"
)

plt.xticks(x, methods)
plt.ylabel("Measurement Distribution Fidelity")
plt.title("Windowed vs Genetic QEC Post-selection")
plt.ylim(0.97, 1.001)
plt.legend()
plt.grid(axis="y", alpha=0.3)

plt.tight_layout()
plt.show()
```




```
# ========================================
# COMPARISON: 1,000 SHOTS vs 10,000 SHOTS
# ========================================

comparison = pd.DataFrame({
    "Metric": [
        "Acceptance Rate (%)",
        "Fidelity Before",
        "Fidelity After",
        "Fidelity Improvement (pp)"
    ],

    "Windowed (1000 shots)": [
        results_df["Windowed Acceptance (%)"].mean(),
        results_df["Windowed Fidelity Before"].mean(),
        results_df["Windowed Fidelity After"].mean(),
        results_df["Windowed Improvement"].mean()
    ],

    "Genetic (1000 shots)": [
        results_df["Genetic Acceptance (%)"].mean(),
        results_df["Genetic Fidelity Before"].mean(),
        results_df["Genetic Fidelity After"].mean(),
        results_df["Genetic Improvement"].mean()
    ],

    "Windowed (10000 shots)": [
        results_10000_df["Windowed Acceptance (%)"].mean(),
        results_10000_df["Windowed Fidelity Before"].mean(),
        results_10000_df["Windowed Fidelity After"].mean(),
        results_10000_df["Windowed Improvement (pp)"].mean()
    ],

    "Genetic (10000 shots)": [
        results_10000_df["Genetic Acceptance (%)"].mean(),
        results_10000_df["Genetic Fidelity Before"].mean(),
        results_10000_df["Genetic Fidelity After"].mean(),
        results_10000_df["Genetic Improvement (pp)"].mean()
    ]
})

print("===== FINAL EXPERIMENT COMPARISON =====")
display(comparison.round(6))
```

```
===== FINAL EXPERIMENT COMPARISON =====
```

	Metric	Windowed (1000 shots)	Genetic (1000 shots)	Windowed (10000 shots)	Genetic (10000 shots)
0	Acceptance Rate (%)	99.100000	92.600000	98.700000	93.835500
1	Fidelity Before	0.976025	0.976041	0.978807	0.978906
2	Fidelity After	0.976890	0.997570	0.979136	0.999358
3	Fidelity Improvement (pp)	0.086554	2.152844	0.032898	2.045235

```
import matplotlib.pyplot as plt
import numpy as np

methods = ["Windowed", "Genetic"]

# 1. Fidelity comparison
before = [0.978807, 0.978906]
after = [0.979136, 0.999358]

x = np.arange(2)
width = 0.35

plt.figure(figsize=(8, 5))
plt.bar(x - width/2, before, width, label="Before")
plt.bar(x + width/2, after, width, label="After")

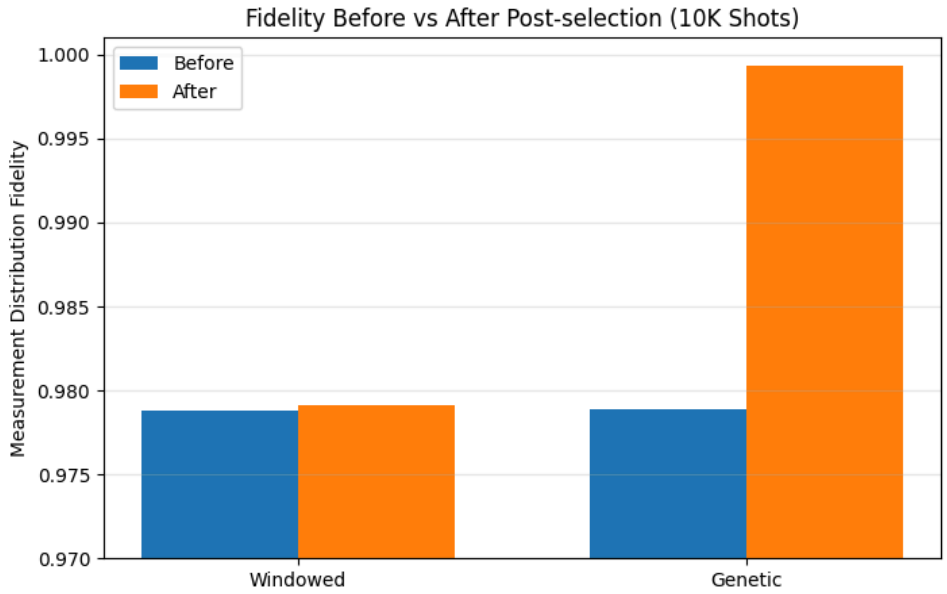
plt.xticks(x, methods)
plt.ylabel("Measurement Distribution Fidelity")
plt.title("Fidelity Before vs After Post-selection (10K Shots)")
plt.ylim(0.97, 1.001)
plt.legend()
plt.grid(axis="y", alpha=0.3)
plt.show()

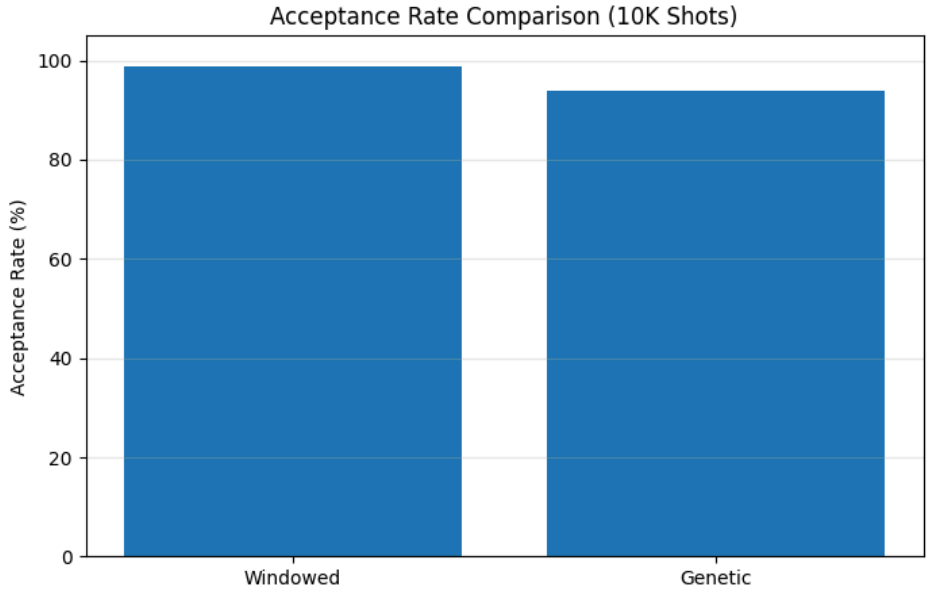

# 2. Acceptance rate comparison
acceptance = [98.70, 93.8355]

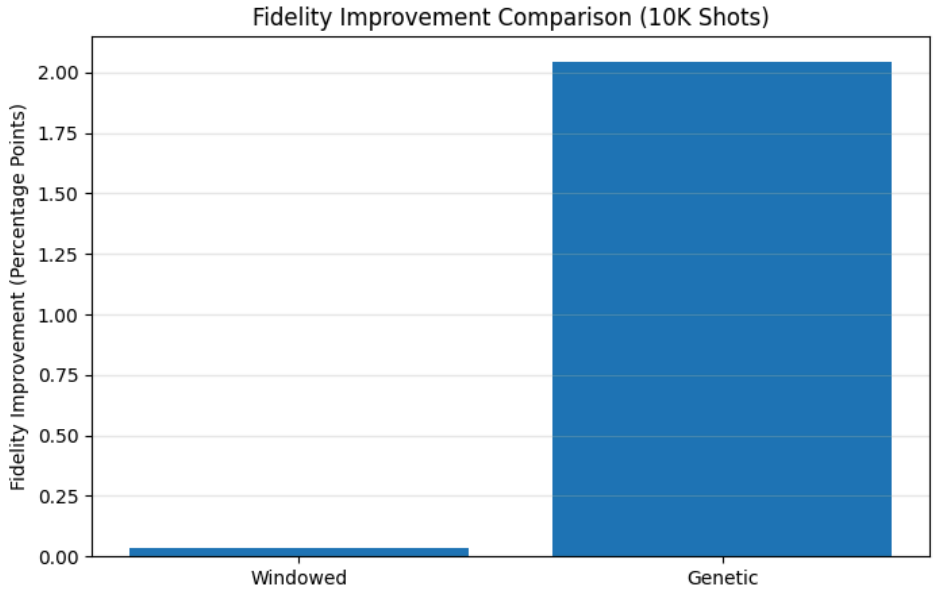
plt.figure(figsize=(8, 5))
plt.bar(methods, acceptance)
plt.ylabel("Acceptance Rate (%)")
plt.title("Acceptance Rate Comparison (10K Shots)")
plt.ylim(0, 105)
plt.grid(axis="y", alpha=0.3)
plt.show()


# 3. Fidelity improvement comparison
improvement = [0.032898, 2.045235]

plt.figure(figsize=(8, 5))
plt.bar(methods, improvement)
plt.ylabel("Fidelity Improvement (Percentage Points)")
plt.title("Fidelity Improvement Comparison (10K Shots)")
plt.grid(axis="y", alpha=0.3)
plt.show()
```








```
# ========================================
# NO-QEC BASELINE: 10,000 SHOTS, 20 SEEDS
# ========================================
```

```
baseline_circuit = checked_variants[0].circuit

baseline_results = []

SHOTS = 10000

for seed in range(20):

    result = simulator.run(
        baseline_circuit,
        shots=SHOTS,
        seed_simulator=seed
    ).result()

    counts = result.get_counts()

    # Extract the three data qubits
    data_counts = marginal_counts(
        counts,
        indices=[0, 1, 2]
    )

    fidelity = calculate_fidelity(
        data_counts,
        true_ideal_data_counts
    )

    baseline_results.append({
        "Seed": seed,
        "Acceptance Rate (%)": 100.0,
        "Fidelity": fidelity
    })

baseline_df = pd.DataFrame(baseline_results)

print("===== NO-QEC BASELINE RESULTS =====")
display(baseline_df.round(6))

print("\n===== BASELINE STATISTICS =====")

print(
    "Mean Fidelity:",
    baseline_df["Fidelity"].mean()
)

print(
    "Standard Deviation:",
    baseline_df["Fidelity"].std()
)
```

```
===== NO-QEC BASELINE RESULTS =====
```

	Seed	Acceptance Rate (%)	Fidelity
0	0	100.0	0.977391
1	1	100.0	0.977390
2	2	100.0	0.977393
3	3	100.0	0.977393
4	4	100.0	0.977393
5	5	100.0	0.977395
6	6	100.0	0.977395
7	7	100.0	0.977395
8	8	100.0	0.977395
9	9	100.0	0.977395
10	10	100.0	0.977395
11	11	100.0	0.977394
12	12	100.0	0.977394
13	13	100.0	0.977394
14	14	100.0	0.977394
15	15	100.0	0.977394
16	16	100.0	0.977394
17	17	100.0	0.977394
18	18	100.0	0.977391
19	19	100.0	0.977392

```
===== BASELINE STATISTICS =====
Mean Fidelity: 0.9773933381362292
Standard Deviation: 1.4554958471777583e-06
```

```
# ========================================
# FINAL THREE-METHOD COMPARISON
# 10,000 SHOTS | 20 SEEDS
# ========================================

final_comparison = pd.DataFrame({

    "Method": [
        "No QEC",
        "Windowed",
        "Genetic"
    ],

    "Mean Acceptance (%)": [
        baseline_df["Acceptance Rate (%)"].mean(),
        results_10000_df["Windowed Acceptance (%)"].mean(),
        results_10000_df["Genetic Acceptance (%)"].mean()
    ],

    "Mean Fidelity": [
        baseline_df["Fidelity"].mean(),
        results_10000_df["Windowed Fidelity Before"].mean(),
        results_10000_df["Genetic Fidelity Before"].mean()
    ],

    "Fidelity After Post-selection": [
        baseline_df["Fidelity"].mean(),
        results_10000_df["Windowed Fidelity After"].mean(),
        results_10000_df["Genetic Fidelity After"].mean()
    ]
})

print("===== FINAL THREE-METHOD RESULTS =====")
display(final_comparison.round(8))
```

```
===== FINAL THREE-METHOD RESULTS =====
```

	Method	Mean Acceptance (%)	Mean Fidelity	Fidelity After Post-selection
0	No QEC	100.0000	0.977393	0.977393
1	Windowed	98.7000	0.978807	0.979136
2	Genetic	93.8355	0.978906	0.999358

```
import matplotlib.pyplot as plt

methods = ["No QEC", "Windowed", "Genetic"]

fidelities = [
    baseline_df["Fidelity"].mean(),
    results_10000_df["Windowed Fidelity After"].mean(),
    results_10000_df["Genetic Fidelity After"].mean()
]

acceptance = [
    baseline_df["Acceptance Rate (%)"].mean(),
    results_10000_df["Windowed Acceptance (%)"].mean(),
    results_10000_df["Genetic Acceptance (%)"].mean()
]

# Graph 1: Fidelity comparison
plt.figure(figsize=(9, 5))

plt.bar(methods, fidelities)

plt.ylabel("Measurement Distribution Fidelity")
plt.title("No QEC vs Windowed vs Genetic")
plt.ylim(0.97, 1.001)
plt.grid(axis="y", alpha=0.3)

for i, value in enumerate(fidelities):
    plt.text(i, value + 0.0003, f"{value:.6f}", ha="center")

plt.tight_layout()
plt.show()

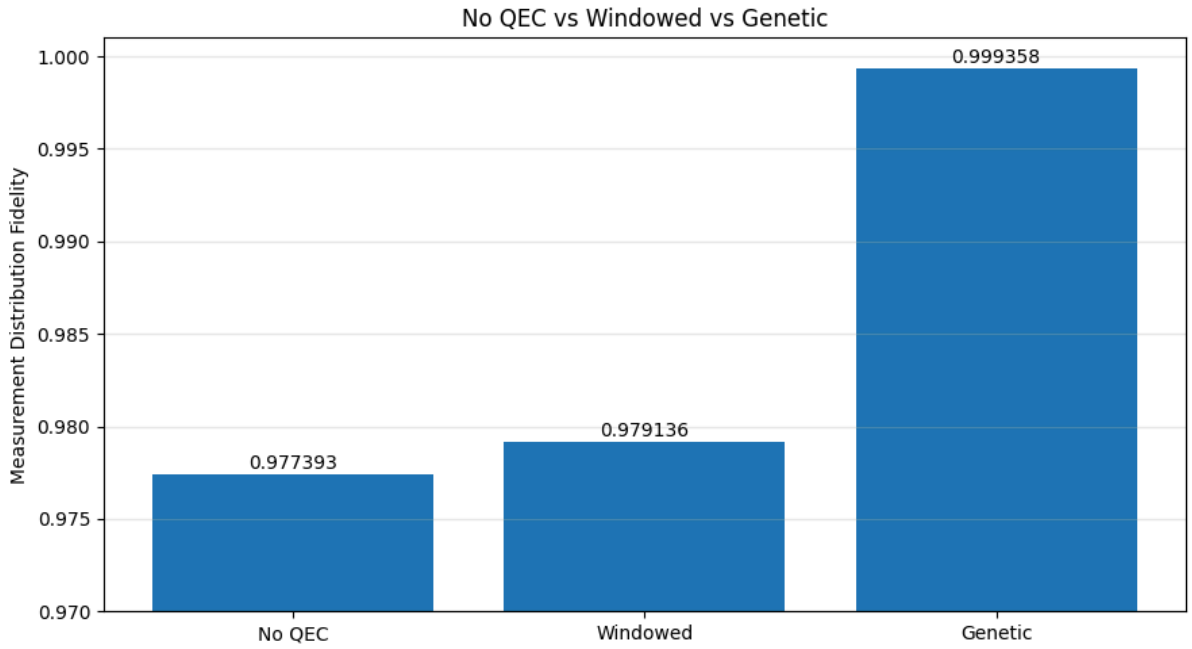

# Graph 2: Acceptance rate comparison
plt.figure(figsize=(9, 5))

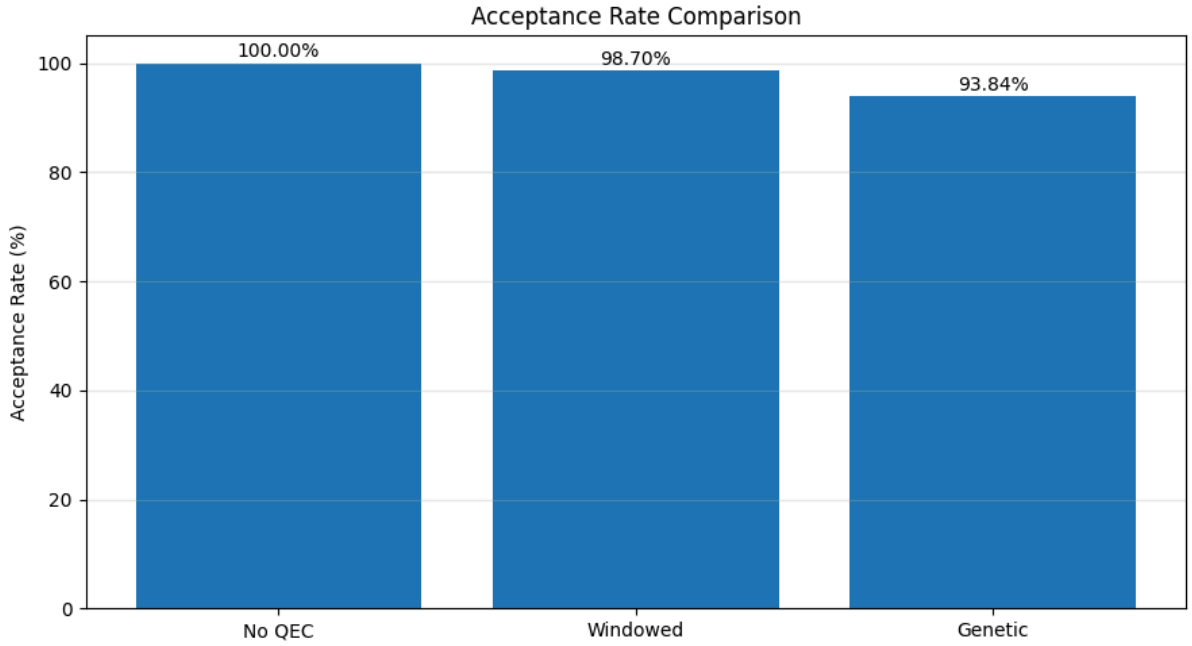
plt.bar(methods, acceptance)

plt.ylabel("Acceptance Rate (%)")
plt.title("Acceptance Rate Comparison")
plt.ylim(0, 105)
plt.grid(axis="y", alpha=0.3)

for i, value in enumerate(acceptance):
    plt.text(i, value + 1, f"{value:.2f}%", ha="center")

plt.tight_layout()
plt.show()
```






```
# ========================================
# CIRCUIT STRUCTURE VALIDATION
# ========================================

circuits = {
    "Original Clifford Circuit": clifford_qec,
    "No QEC": baseline_circuit,
    "Windowed": windowed_circuit,
    "Genetic": genetic_circuit
}

for name, circuit in circuits.items():

    print("\n==============================")
    print(name)
    print("==============================")

    print("Qubits:", circuit.num_qubits)
    print("Classical bits:", circuit.num_clbits)
    print("Circuit depth:", circuit.depth())
    print("Gate count:", circuit.size())
```

```
    print("\nOperation counts:")
    print(circuit.count_ops())
```

```
==============================
Original Clifford Circuit
==============================
Qubits: 3
Classical bits: 3
Circuit depth: 5
Gate count: 9

Operation counts:
OrderedDict({'measure': 3, 'h': 2, 'cx': 2, 's': 2, 'barrier': 1})

==============================
No QEC
==============================
Qubits: 3
Classical bits: 3
Circuit depth: 5
Gate count: 9

Operation counts:
OrderedDict({'measure': 3, 'h': 2, 'cx': 2, 's': 2})

==============================
Windowed
==============================
Qubits: 4
Classical bits: 4
Circuit depth: 25
Gate count: 32

Operation counts:
OrderedDict({'h': 12, 's': 11, 'cx': 5, 'measure': 4})

==============================
Genetic
==============================
Qubits: 6
Classical bits: 6
Circuit depth: 10
Gate count: 31

Operation counts:
OrderedDict({'h': 18, 'measure': 6, 'cx': 5, 's': 2})
```

```
# ========================================
# GATE OVERHEAD COMPARISON
# ========================================

import pandas as pd

circuit_data = {
    "Method": ["No QEC", "Windowed", "Genetic"],
    "Total Qubits": [3, 4, 6],
    "Ancilla Qubits": [0, 1, 3],
    "Circuit Depth": [5, 25, 10],
    "Gate Count": [9, 32, 31]
}

overhead_df = pd.DataFrame(circuit_data)

baseline_gates = 9
baseline_depth = 5

overhead_df["Gate Overhead (%)"] = (
    (overhead_df["Gate Count"] - baseline_gates)
    / baseline_gates
) * 100

overhead_df["Depth Overhead (%)"] = (
    (overhead_df["Circuit Depth"] - baseline_depth)
    / baseline_depth
) * 100

print("===== CIRCUIT OVERHEAD ANALYSIS =====")
```

```
display(overhead_df.round(2))
```

```
===== CIRCUIT OVERHEAD ANALYSIS =====
```

	Method	Total Qubits	Ancilla Qubits	Circuit Depth	Gate Count	Gate Overhead (%)	Depth Overhead (%)
0	No QEC	3	0	5	9	0.00	0.0
1	Windowed	4	1	25	32	255.56	400.0
2	Genetic	6	3	10	31	244.44	100.0

```
import matplotlib.pyplot as plt

methods = ["No QEC", "Windowed", "Genetic"]

fidelity = [
    baseline_df["Fidelity"].mean(),
    results_10000_df["Windowed Fidelity After"].mean(),
    results_10000_df["Genetic Fidelity After"].mean()
]

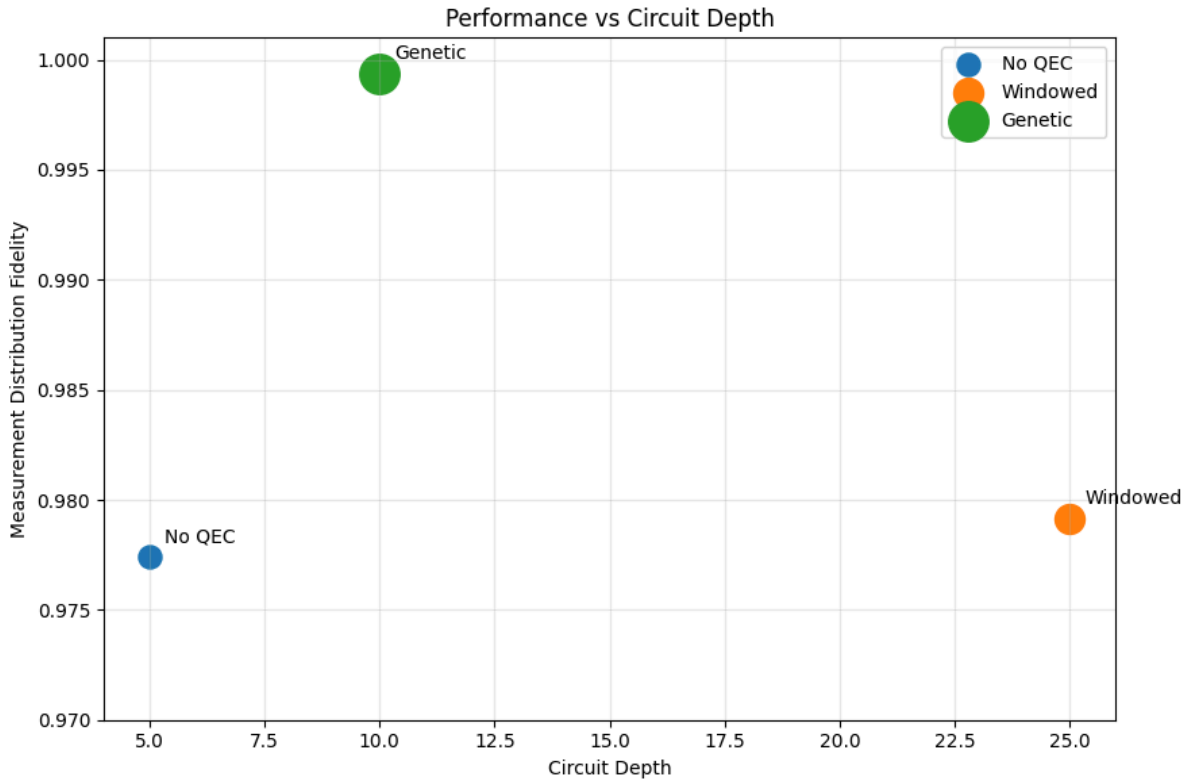
depth = [5, 25, 10]
ancillas = [0, 1, 3]

plt.figure(figsize=(9, 6))

for i in range(3):
    plt.scatter(
        depth[i],
        fidelity[i],
        s=150 + ancillas[i] * 100,
        label=methods[i]
    )

    plt.annotate(
        methods[i],
        (depth[i], fidelity[i]),
        xytext=(8, 8),
        textcoords="offset points"
    )

plt.xlabel("Circuit Depth")
plt.ylabel("Measurement Distribution Fidelity")
plt.title("Performance vs Circuit Depth")
plt.ylim(0.97, 1.001)
plt.grid(alpha=0.3)
plt.legend()
plt.tight_layout()
plt.show()
```




```
# ========================================
# ORIGINAL DATA CIRCUIT VALIDATION
# ========================================

print("===== ORIGINAL CIRCUIT =====")
print(clifford_qec.draw())

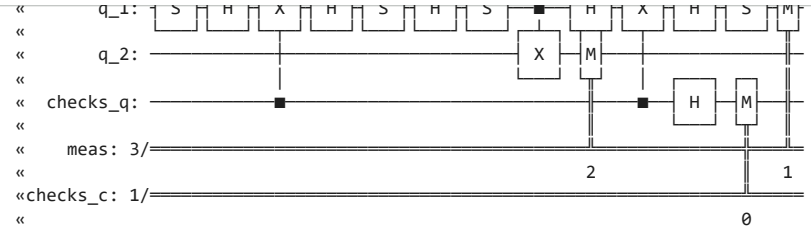
print("\n===== NO-QEC CIRCUIT =====")
print(baseline_circuit.draw())

print("\n===== WINDOWED CIRCUIT =====")
print(windowed_circuit.draw())

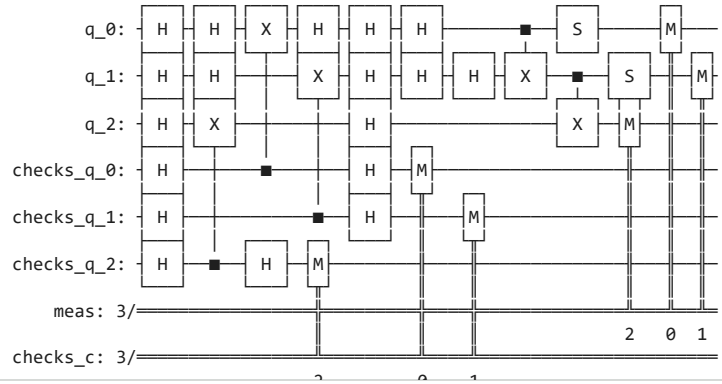
print("\n===== GENETIC CIRCUIT =====")
print(genetic_circuit.draw())
```

```
===== NO-QEC CIRCUIT =====
   q_0: ┤ H ├──■──┤ S ├──────┤ M ├──────
   q_1: ┤ H ├┤ X ├──■──┤ S ├──╫──┤ M ├──
   q_2: ──────┤ X ├──┤ M ├──╫──╫──╫──
meas: 3/═══════════════════╩══╩══╩══
                           2  0  1

===== WINDOWED CIRCUIT =====
        ┌───┐                                                  ┌───┐      ┌───┐»
   q_0: ┤ H ├──────────────────────────────────────────────────┤ S ├──────┤ M ├»
        ├───┤┌───┐┌───┐┌───┐┌───┐┌───┐┌───┐┌───┐┌───┐┌───┐┌─╫─┐┌─╫─┐»
   q_1: ┤ H ├┤ X ├┤ H ├┤ H ├┤ X ├┤ S ├┤ S ├┤ S ├┤ H ├┤ S ├┤ S ├»
        └───┘└───┘└───┘└───┘└───┘└───┘└───┘└───┘└───┘└───┘└─╫─┘»
   q_2: ──────────────────────────────────────────────────────╫──»
                                                             ╩  »
```




```
===== GENETIC CIRCUIT =====
```




```
print("===== WINDOWED VARIANT =====")

print("Type:", type(checked_variants[1]))
print("Available attributes:")
print(dir(checked_variants[1]))

print("\n===== GENETIC VARIANT =====")

print("Type:", type(genetic_variants[3]))
print("Available attributes:")
print(dir(genetic_variants[3]))
```

```
rcuit', 'cost', 'cost_metric', 'dope', 'estimate_fault_rates', 'get_postselection_method', 'target_qubits', 'uncovered_paulis']


rcuit', 'cost', 'cost_metric', 'dope', 'estimate_fault_rates', 'get_postselection_method', 'target_qubits', 'uncovered_paulis']
```

```
# POST-SELECTION VALIDATION

windowed_variant = checked_variants[1]
genetic_variant = genetic_variants[3]

print("===== WINDOWED VARIANT =====")
print("Target qubits:", windowed_variant.target_qubits)
print("Check qubits:", windowed_variant.check_qubits)
print("Check support:", windowed_variant.check_support)
print("Post-selection method:", windowed_variant.get_postselection_method())

print("\n===== GENETIC VARIANT =====")
print("Target qubits:", genetic_variant.target_qubits)
print("Check qubits:", genetic_variant.check_qubits)
print("Check support:", genetic_variant.check_support)
print("Post-selection method:", genetic_variant.get_postselection_method())
```

```
===== WINDOWED VARIANT =====
Target qubits: (1,)
Check qubits: (3,)
Check support: ((3,),)
Post-selection method: <function CheckedCircuit.get_postselection_method.<locals>._aux at 0x7da5010ac680>

===== GENETIC VARIANT =====
Target qubits: (0, 1, 2)
Check qubits: (3, 4, 5)
Check support: ((0, 3), (0, 1, 4), (1, 2, 5))
Post-selection method: <function CheckedCircuit.get_postselection_method.<locals>._aux at 0x7da5010ac680>
```

```
import numpy as np

# Get post-selection functions
windowed_ps = checked_variants[1].get_postselection_method()
genetic_ps = genetic_variants[3].get_postselection_method()

# Function to calculate acceptance and rejection
def calculate_acceptance(counts, postselection_fn):
    total_shots = sum(counts.values())
    accepted_shots = 0

    for bitstring, count in counts.items():
        syndrome = postselection_fn(bitstring)

        if np.all(np.asarray(syndrome) == 0):
            accepted_shots += count

    rejected_shots = total_shots - accepted_shots
    acceptance_rate = (accepted_shots / total_shots) * 100

    return accepted_shots, rejected_shots, acceptance_rate


# Use your existing measurement count dictionaries
# Replace these names only if your notebook uses different names.

windowed_accepted, windowed_rejected, windowed_rate = calculate_acceptance(
    windowed_counts, windowed_ps
)

genetic_accepted, genetic_rejected, genetic_rate = calculate_acceptance(
    genetic_counts, genetic_ps
)

print("===== WINDOWED =====")
print("Accepted shots:", windowed_accepted)
print("Rejected shots:", windowed_rejected)
print("Acceptance rate:", round(windowed_rate, 2), "%")

print("\n===== GENETIC =====")
print("Accepted shots:", genetic_accepted)
print("Rejected shots:", genetic_rejected)
print("Acceptance rate:", round(genetic_rate, 2), "%")
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
/tmp/ipykernel_528/3885454534.py in <cell line: 0>()
     26
     27 windowed_accepted, windowed_rejected, windowed_rate = calculate_acceptance(
---> 28     windowed_counts, windowed_ps
     29 )
     30

NameError: name 'windowed_counts' is not defined
```

Next steps: Explain error

```
print("Available count variables:")

for name, value in globals().copy().items():
    if "count" in name.lower() and isinstance(value, dict):
        print(name, "->", type(value), "->", len(value), "entries")
```

```
Available count variables:
counts -> <class 'qiskit.result.counts.Counts'> -> 8 entries
counts_postselected -> <class 'dict'> -> 8 entries
ideal_counts -> <class 'qiskit.result.counts.Counts'> -> 8 entries
current_counts -> <class 'qiskit.result.counts.Counts'> -> 11 entries
accepted_counts -> <class 'dict'> -> 8 entries
genetic_counts -> <class 'qiskit.result.counts.Counts'> -> 27 entries
genetic_postselected_counts -> <class 'dict'> -> 5 entries
genetic_data_counts -> <class 'dict'> -> 8 entries
genetic_postselected_data_counts -> <class 'dict'> -> 5 entries
ideal_data_counts -> <class 'dict'> -> 8 entries
```

```
true_ideal_counts -> <class 'qiskit.result.counts.Counts'> -> 4 entries
true_ideal_data_counts -> <class 'dict'> -> 4 entries
w_counts -> <class 'qiskit.result.counts.Counts'> -> 14 entries
g_counts -> <class 'qiskit.result.counts.Counts'> -> 40 entries
data_counts -> <class 'dict'> -> 8 entries
```

```
import numpy as np

# Get post-selection functions
windowed_ps = checked_variants[1].get_postselection_method()
genetic_ps = genetic_variants[3].get_postselection_method()

# Calculate acceptance and rejection
def calculate_acceptance(counts, postselection_fn):
    total_shots = sum(counts.values())
    accepted_shots = 0

    for bitstring, count in counts.items():
        syndrome = postselection_fn(bitstring)

        if np.all(np.asarray(syndrome) == 0):
            accepted_shots += count

    rejected_shots = total_shots - accepted_shots
    acceptance_rate = (accepted_shots / total_shots) * 100

    return accepted_shots, rejected_shots, acceptance_rate


# Windowed results
w_accepted, w_rejected, w_rate = calculate_acceptance(
    w_counts, windowed_ps
)

# Genetic results
g_accepted, g_rejected, g_rate = calculate_acceptance(
    genetic_counts, genetic_ps
)

print("===== WINDOWED =====")
print("Total shots:", sum(w_counts.values()))
print("Accepted shots:", w_accepted)
print("Rejected shots:", w_rejected)
print("Acceptance rate:", round(w_rate, 2), "%")

print("\n===== GENETIC =====")
print("Total shots:", sum(genetic_counts.values()))
print("Accepted shots:", g_accepted)
print("Rejected shots:", g_rejected)
print("Acceptance rate:", round(g_rate, 2), "%")
```

```
===== WINDOWED =====
Total shots: 10000
Accepted shots: 9870
Rejected shots: 130
Acceptance rate: 98.7 %

===== GENETIC =====
Total shots: 1000
Accepted shots: 942
Rejected shots: 58
Acceptance rate: 94.2 %
```

```
print("===== FIDELITY VARIABLES =====")

for name, value in globals().copy().items():
    if "fidelity" in name.lower():
        print(name, "=", value)
```

```
===== FIDELITY VARIABLES =====
classical_fidelity = <function classical_fidelity at 0x7da52831d760>
state_fidelity = <function state_fidelity at 0x7da53c0c1800>
calculate_classical_fidelity = <function calculate_classical_fidelity at 0x7da51f201260>
fidelity_before = [np.float64(0.9788070480660801), np.float64(0.978905779056251)]
fidelity_after = [np.float64(0.9791360322657316), np.float64(0.9993581247492976)]
calculate_fidelity = <function calculate_fidelity at 0x7da5013631a0>
fidelity = [np.float64(0.9773933381362292), np.float64(0.9791360322657316), np.float64(0.9993581247492976)]
```

```
print("===== FIDELITY COMPARISON =====")

print("\nWINDOWED METHOD")
print("Fidelity Before:", round(float(fidelity_before[0]), 6))
print("Fidelity After:", round(float(fidelity_after[0]), 6))
print("Improvement:", round(
    float((fidelity_after[0] - fidelity_before[0]) * 100), 6
), "percentage points")

print("\nGENETIC METHOD")
print("Fidelity Before:", round(float(fidelity_before[1]), 6))
print("Fidelity After:", round(float(fidelity_after[1]), 6))
print("Improvement:", round(
    float((fidelity_after[1] - fidelity_before[1]) * 100), 6
), "percentage points")
```

```
===== FIDELITY COMPARISON =====

WINDOWED METHOD
Fidelity Before: 0.978807
Fidelity After: 0.979136
Improvement: 0.032898 percentage points

GENETIC METHOD
Fidelity Before: 0.978906
Fidelity After: 0.999358
Improvement: 2.045235 percentage points
```

```
import matplotlib.pyplot as plt
import numpy as np

methods = ["No QEC", "Windowed", "Genetic"]

fidelity_values = [
    0.977393,
    float(fidelity_after[0]),
    float(fidelity_after[1])
]

plt.figure(figsize=(9, 5))

bars = plt.bar(methods, fidelity_values)

plt.ylim(0.95, 1.01)
plt.ylabel("Classical Measurement Fidelity")
plt.title("Comparison of QEC Methods")

for bar, value in zip(bars, fidelity_values):
    plt.text(
        bar.get_x() + bar.get_width() / 2,
        bar.get_height() + 0.001,
        f"{value:.6f}",
        ha="center"
    )

plt.grid(axis="y", alpha=0.3)
plt.show()
```